



Service-basierte Komponenten- Architekturen (SEKA)

Kapitel 3 – Modellierung und Entwicklung von komponentenbasierten Software-Architekturen

**Vorlesung im Studiengang Master Informatik
Sommersemester 2026**

Prof. Dr. Sascha Alda
(sascha.alda@h-brs.de)



Kapitel	Thema	
1	Organisation, Einführung in Software-Architekturen	✓
2	Software-Komponenten und Microservices – Grundlagen, Methoden und ein erster Technologie-Überblick	✓
3	Modellierung und Entwicklung von komponentenbasierten Software-Architekturen	
4	Containerisierung und Integration von Microservices – moderne Konzepte und Technologien	
5	Muster für komponentenbasierte Unternehmensapplikationen	
6	Methoden und Technologien zur Orchestration von Services – (Camunda vs. MCP)	
7	Grundlagen und Methoden zur Migration (am Beispiel Java EE)	
	02.06.2026: Gastvortrag der FA. Capgemini AG (Thomas Rath) <i>„Microservices und Docker in der Praxis im KI-Zeitalter“</i> Weitere Gastvorlesung in Planung	



Kapitel 3: Modellierung und Entwicklung von komponentenbasierten Software-Architekturen

1	Vorgehensmodelle zur Entwicklung von komponentenbasierten Architekturen und zur Bewertung von Komponenten	
2	Semi-formale Modellierung von Architekturen: Das 4 Sichten-Modell	
3	Weitere Formen zur Dokumentation und formalen Modellierung von Software-Architekturen (arc42, ADL)	
4	Referenz-Architekturen für komponentenbasierte Architekturen	
5	Zusammenfassung und Ausblick	

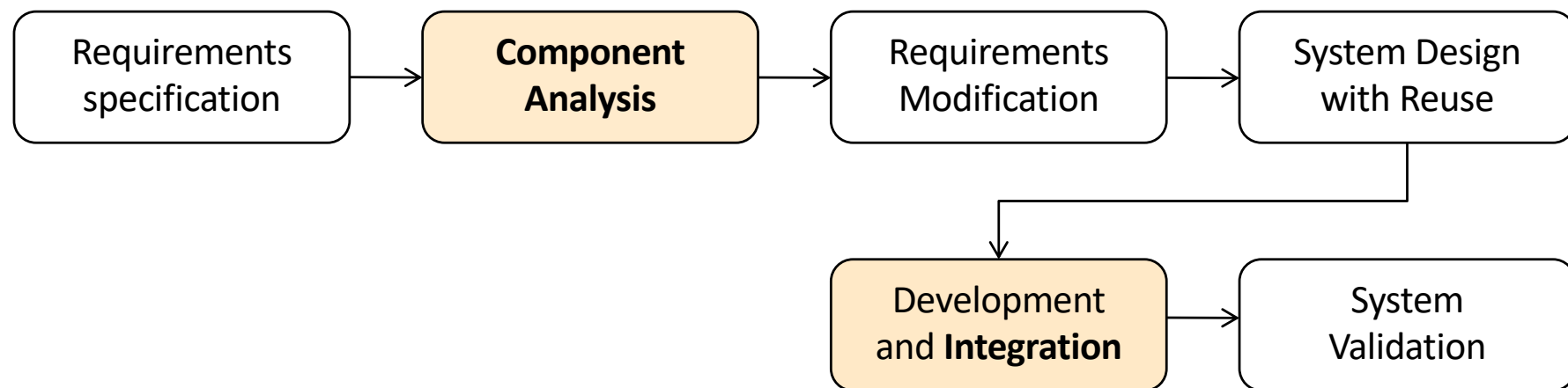


- Fokussierung auf **Wiederverwendung** von Komponenten → Component-based Software Engineering (CBSE)
- Zu unterscheiden:
 - Entwicklungsprozess zur Entwicklung einer komponentenbasierten Applikation (d.h. das System als Ganzes)
 - Entwicklungsprozess zur Entwicklung individueller Komponenten, möglicherweise durch eine andere Entwicklungsorganisation
- Zentrale zusätzliche Aktivität: **Analyse und Bewertung** von Komponenten (*component assessment*)
- Kein Standard vorhanden in der Industrie, wissenschaftliche Ansätze liefern jedoch gute Ansätze!
 - Fokus auf traditionelle Prozessmodelle, eine Überführung auf agile und iterative Modelle ist aber durchführbar

Phasenmodell zur Entwicklung einer komponentenbasierten Applikation

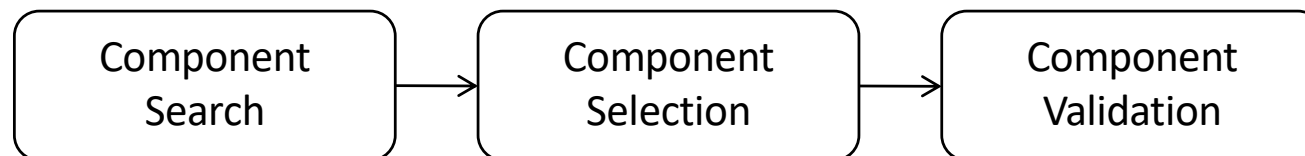


- Sommerville ergänzt einen typischen Entwicklungsprozess um Phasen und Aktivitäten zur **Analyse und Integration** von Software-Komponenten (Sommerville 2008)





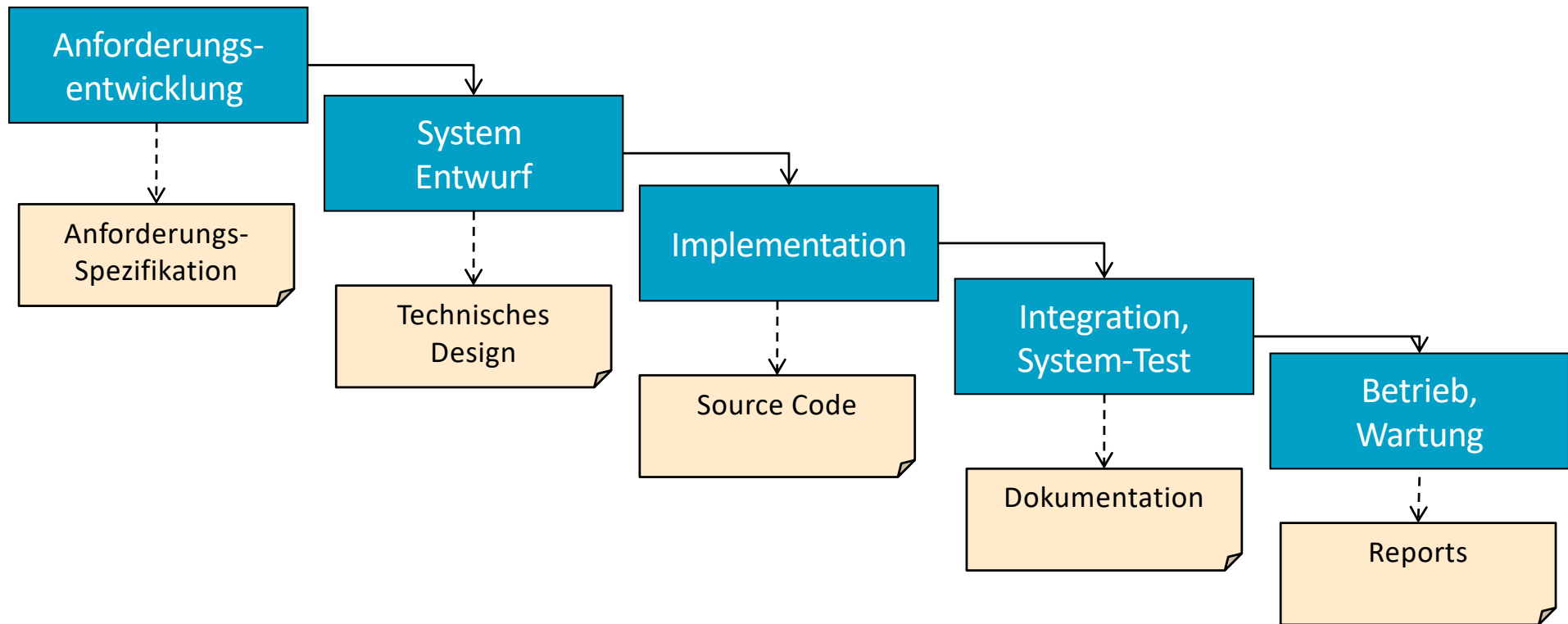
- Component Analysis (oder: *Assessment*)
 - Suche nach Komponenten, die (Teile) einer Anforderungsspezifikation oder einer Schnittstellenspezifikation erfüllen
 - Meistens *kein* exakter Match möglich (inkompatible Schnittstellen oder Funktionalität)
 - Component Reasoning: automatisierte Klassifizierung von Komponenten z.B. über Ontologien (vgl. Mahmood, 2007), (Pahl, 2003); KI-basierte Tools (LLM)
 - Priorität bei der Suche:
 - Lokale Suche (z.B. Repository, verwandte Projekte)
 - Suche bei vertrauten Anbietern (*trusted Suppliers*)
 - Suche bei anderen Anbietern
- Möglicher Subprozess:





- Ausgehend von einer soliden Anforderungsspezifikation weist der CBSE neue Phasen auf, die wie folgt zu realisieren sind:
- Requirements Modification
 - Wiederholte Analyse der Anforderungen auf Basis der gefundenen Komponenten
 - Berücksichtigung / Überarbeitung der Anforderungsspezifikation unter der Berücksichtigung gefundener Komponenten
- System Design with Reuse
 - Entwicklung der Software-Architektur unter der Berücksichtigung der Komponenten
 - Auswahl eines Komponentenmodells und Laufzeitumgebung (falls nicht durch Rahmenbedingung vorgegeben)
- Development and Integration
 - Integration der Software-Komponenten (Integration und Entwicklung in einer Phase)
 - Software, die nicht als Komponente erworben wurden, wird implementiert

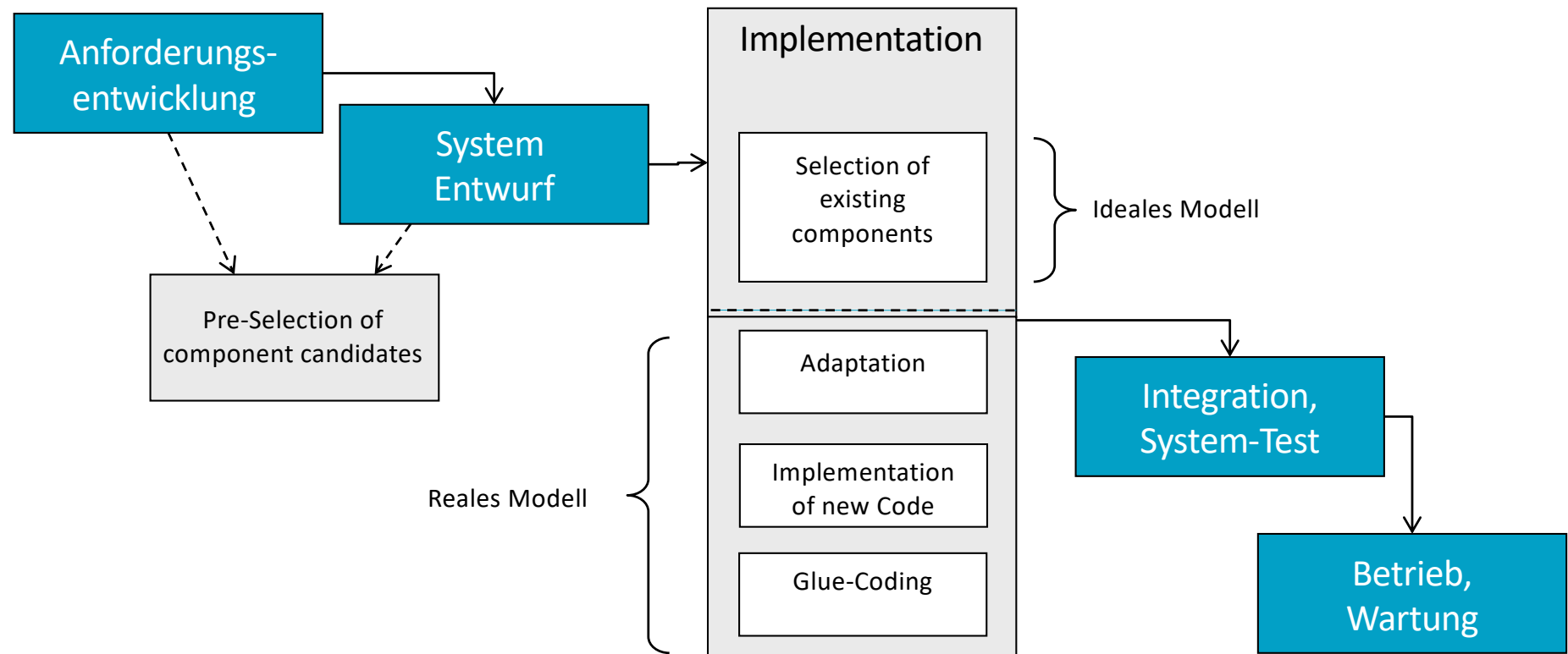
Das Wasserfallmodell (Royce, 1970)



- Die fundamentalen Aktivitäten eines Software-Prozesses sind in einzelne, in sich abgeschlossene Phasen dargestellt
- **Sequentielle** Anordnung und Ausführung der einzelnen Phasen
- Ergebnis einer Phase ist ein Dokument, das von Stakeholdern abgenommen werden muss (Abnahme, engl.: „*sign-off*“)

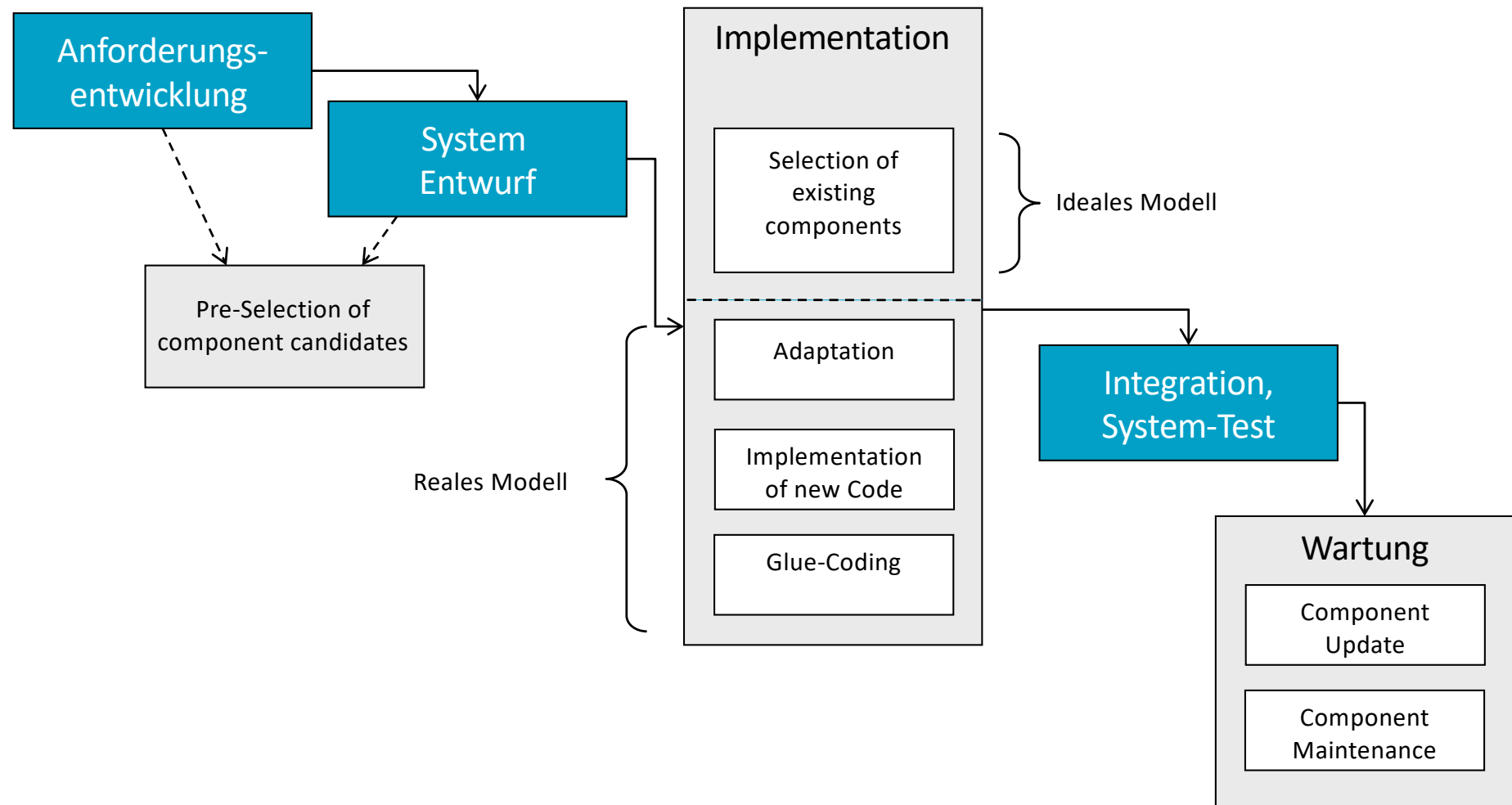


- „Entschlankung“ der Phase **Implementation**, Fokus auf die *reine* Selektierung von existierenden Komponenten (Idealmodell für die Systementwicklung)



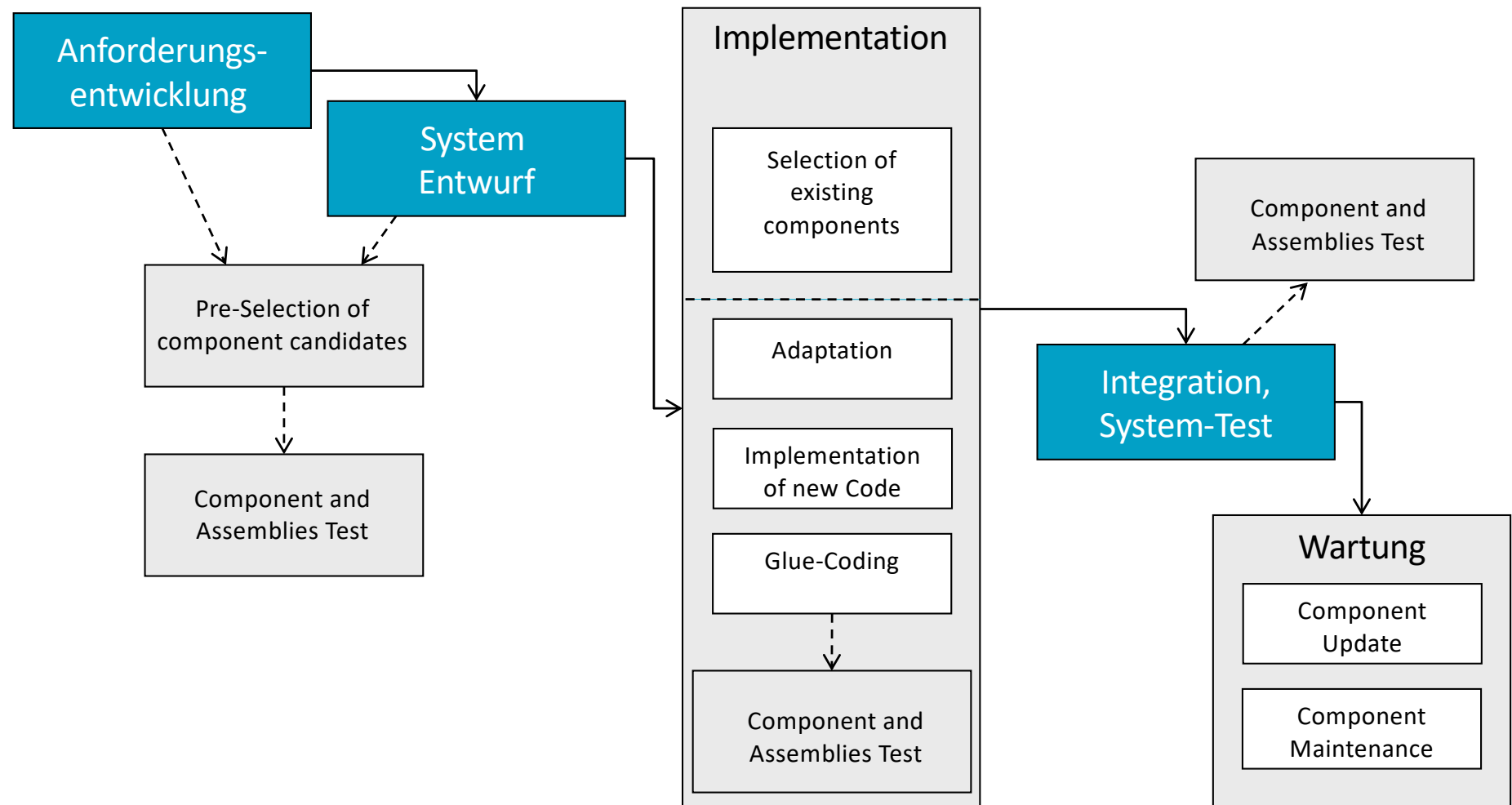


- Weitere Unterstützung in der Phase **Wartung** zur Weiterentwicklung des Systems





- Der **Test von Komponenten** (und Kompositionen) sollte in verschiedenen Phasen des Entwicklungszyklus erfolgen:





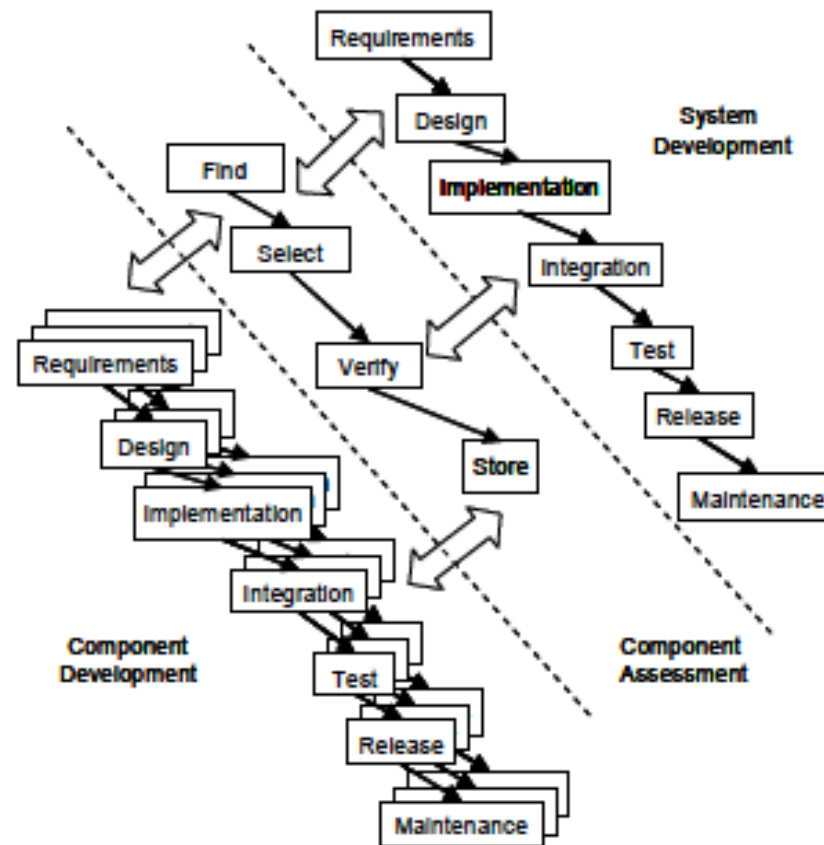
- Anforderungsentwicklung
 - Verschiedene Anforderungen sollten analysiert werden im Hinblick auf eine zukünftige (Wieder-)Verwendung in **noch nicht absehbaren Szenarien**
- Analyse und Design (System-Entwurf)
 - **Festlegung eines Komponentenmodells**, um Annahmen über mögliche Zielsysteme zu machen sowie zur Definition von Diensten (z.B. Security, Transaktionen)
- Test (im allgemeinen)
 - Betrachtung von einer Vielzahl von Einsatzmöglichkeiten (z.B. verschiedene Betriebssysteme)
 - Hohe Anzahl von möglichen Integrationstests

Fazit: bei der Entwicklung einzelner Komponenten müssen eine Vielzahl von „Eventualitäten“ berücksichtigt werden

Quelle: (Crnkovic, 2006)

Integration der verschiedenen Phasen – Fokus: Wiederverwendbarkeit von Komponenten!

- Parallele Durchführung der Prozessmodelle (System und Komponenten). Diese werden begleitet durch einen weiteren **parallelen Prozess zur Bewertung von Komponenten (Component Assessment)**
- Ziel des System-Entwicklers: Fokus auf Wiederverwendbarkeit und Time-To-Market



Quelle: Crnkovic, 2006

Varianten zur Auswahl von Komponenten

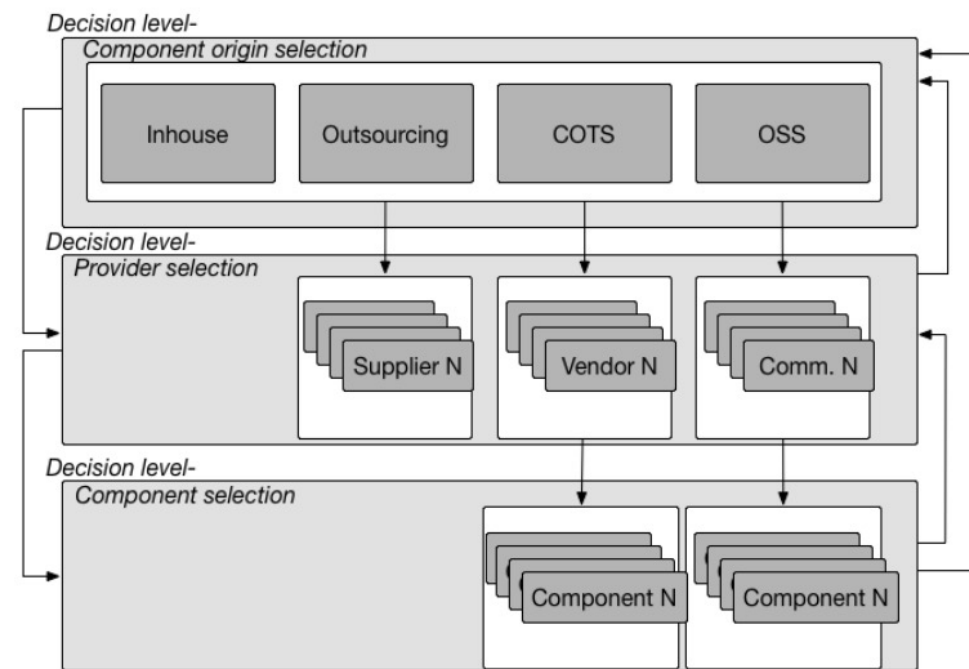
(Component Origin Selection (CSO)) (Petersen, Crnkovic, et al., 2018)



- In-house:
 - Die Komponenten wird **innerhalb der gleichen Firma** entwickelt
 - Mögliche Variante: verteilte Entwicklung auf mehreren Standorten verteilt

- COTS („Commercial-off-the-shelf“):
 - **Kommerzielle** Komponenten sind vorgefertigt
 - Source Code ist nicht verfügbar

- OSS („Open Source Systems“):
 - Komponenten sind vorgefertigt
 - Source Code ist verfügbar
 - Entwicklung in Open Source **Community** (z.B. GitHub)



Prozess(e) zur Auswahl von Komponenten

- Outsource:
 - Entwicklung durch ein Dritt-Unternehmen (Dienstleister) nach Spezifikation durch Unternehmen, **Auslagerung** somit der Entwicklung (Off-Shore, Near-Shore)

Component Assessment – Ein möglicher Ansatz

(Ernst, Kazman und Bianco, 2019)



- Eine (teil-)automatisierte Bewertung und der Vergleich von Komponenten kann über eine **Component Scorecard** erfolgen.
- Adaption einer Balanced Scorecard (Mapping von strategischen Geschäftszielen auf konkrete Metriken, aussagekräftigen Indikatoren und messbaren (SMART) Zielen)
 - Balance Scorecard dienen der Unternehmens-Optimierung, *nicht* dem Vergleich von Zielen
 - Component Scorecard: Fokus auf **Vergleich auf Technologien**
- Prozess für die Bewertung und den Vergleich:
 1. Establish **common criteria** for component selection, based on pre-existing knowledge
 2. Identify **existing tools** which can automatically evaluate these criteria;
 3. Produce a summary **component scorecard**
 4. Aggregate the indicators to produce a final, **cumulative score** per component.
- In der Fallstudie werden drei Kriterien-Bereiche identifiziert:
 - Performance (z.B. Latency, Total Instructions, Memory Utilization, Memory Leaks)
 - Modifizierbarkeit (z.B. Decoupling Level (DL), Propagation Cost (PC))
 - Community Health (z.B. Besitz einer praktikablen Lizenz, Commit-Verhalten)

Component Assessment – Component Scorecard



Quality Attribute	Measurement Tool	Indicator	Component Dlib 19.10	OpenCV 3.3.1
Performance	Instrumentation code	Time 1000 images (ms)	44172	55978
	gperf	Time 1000 images (ms)	47480	58400
	valgrind callgrind	millions of total instructions	490,946	272,331
Memory leaks	Memcheck	bytes possibly lost	288	17,127
Memory Utilization	"	heap usage total megabytes	4,591	1,092
Modifiability	DV8	Decoupling Level	0.51	0.79
	"	Propagation Cost	0.31	0.14
	Understand	SLOC	276,825	783,344
	"	Comment to Code Ratio	0.22	0.31
Security	FlawfinderRaw	hits 3+	162	676
Community Health	CodeMaat	authors > 5 commits	13	234
	"	total commits	7191	18272
	"	authors > 20% commits	1	0
	"	commits last 3 mths	235	686
	"	contribution acceptance rate	0.768	0.585
	"	open_issues	13.8	50.8
	"	first_commit_date/age	May 2008	May 2010 (git history)

Quelle: (Ernst, Kazman und Bianco, 2019)

Tool CodeMaat: <https://github.com/adamtornhill/code-maat>

Decoupling Level (DL): (Mo, Cai, Kazman et al., 2016)



- Beschreibung von möglichen Attributen:

	<i>Attributes</i>	<i>Description</i>
1	Size	Size of the component, e.g. lines of code, memory footprint
2	Longevity prediction	Evolution of the component
3	Cost	Development, license and maintenance cost
4	Level of off-the-shelf fit to product	Functional fitness, i.e., how much component customization is needed
5	Complexity	Code complexity
6	API adequacy	Maturity of external APIs
7	Programming language Performance	Computational performance
8	Access to relevant documentation	Access to documentation
9	Code quality	Availability of automated tests, code review practices.
10	Support of the component	Formal support, channels, active development community
11	Adherence to standards	Follow the rules
12	Other	

- Bewertung durch ca. 150 Probanden

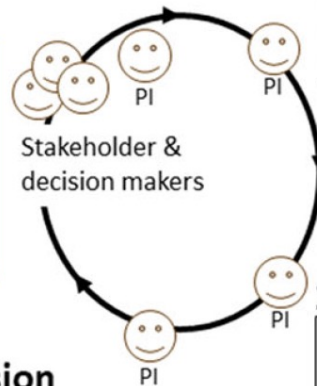
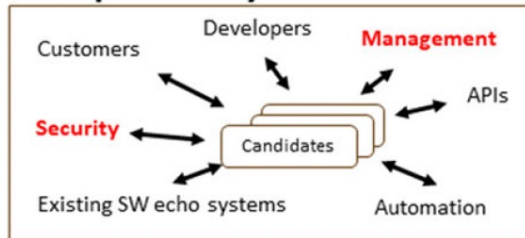
<i>Attributes</i>	<i>Min</i>	<i>Max</i>	<i>Number of respondents</i>	<i>Total points</i>
Cost	0	100	121	3935
Support of the component	0	50	74	1343
Longevity prediction	0	60	71	1513
Level of off-the-shelf fit to product	0	75	62	1499
API adequacy	0	100	56	1195
Access to relevant documentation	0	100	53	1050
Code quality	0	100	53	1280
Adherence to standards	0	50	42	833
Programming language Performance	0	100	41	1112
Complexity	0	50	33	695
Size	0	60	18	435
Other	0	100	18	810

Software Selection Model

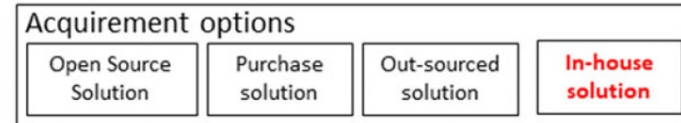
(Bjarnason et al., 2023)



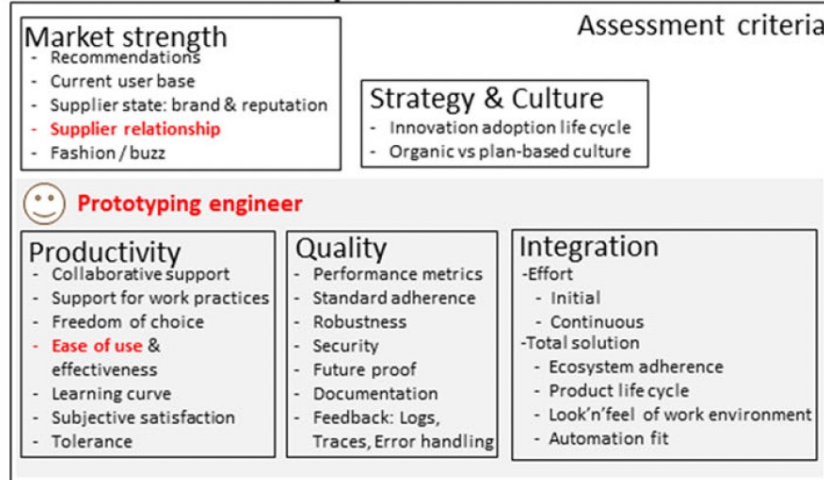
4. Impact analysis



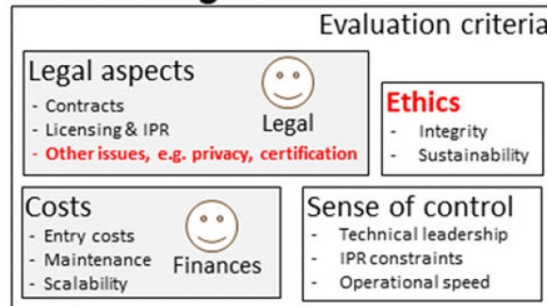
1. Identify solution options



2. Assess solution options



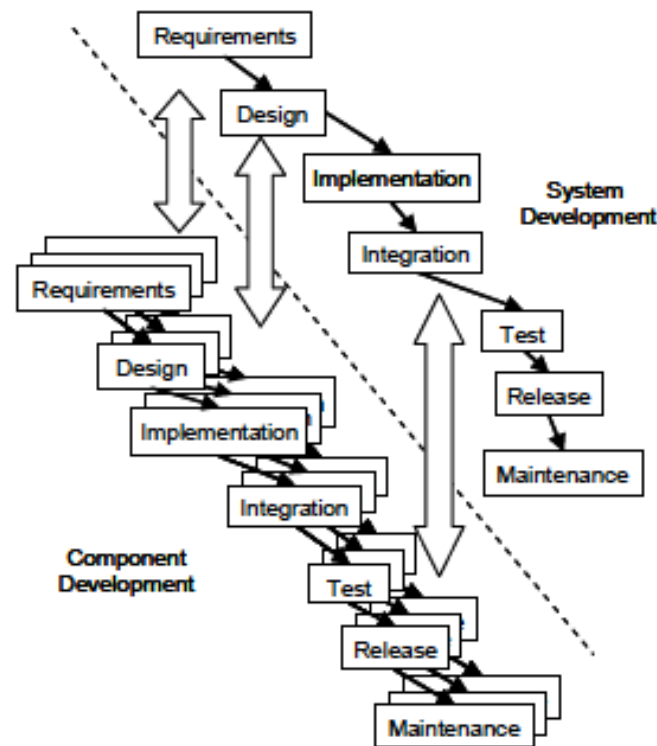
3. Screening decision



Architecture-driven Component Development



- Komponenten werden als Bestandteil der Architektur gedeutet und so auch unmittelbar implementiert.
- Verwendung von Komponentenmodellen **aufgrund ihrer Vorteile** (z.B. Kompositionsfähigkeit, Spezifikation, horizontale Dienste)
- Kein Fokus auf Wiederverwendbarkeit und Time-To-Market – daher kein Assessment!



Quelle: Crnkovic, 2006



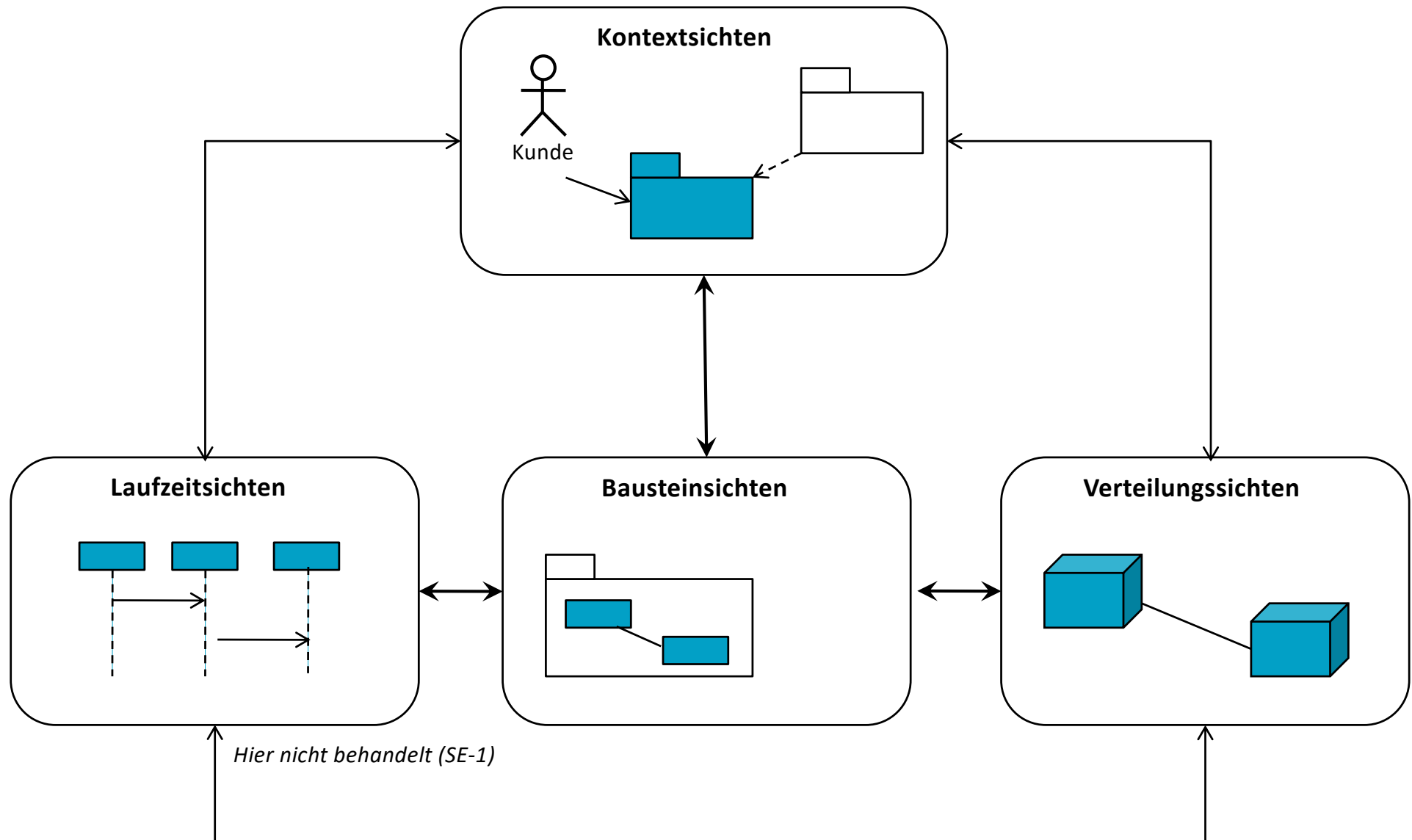
Kapitel 3: Modellierung und Entwicklung von komponentenbasierten Software-Architekturen

1	Vorgehensmodelle zur Entwicklung von komponentenbasierten Architekturen und zur Bewertung von Komponenten	✓
2	Semi-formale Modellierung von Architekturen: Das 4 Sichten-Modell	
3	Weitere Formen zur Dokumentation und formalen Modellierung von Software-Architekturen (arc42, ADL)	
4	Referenz-Architekturen für komponentenbasierte Architekturen	
5	Zusammenfassung und Ausblick	



- Software-Architekten müssen ihre Entwurfsentscheidungen für eine Software-Architektur an verschiedene Stakeholder kommunizieren und dokumentieren
- Eine Diskussion auf Basis von Source-Code ist nahezu unmöglich.
- Die Bereitstellung (Modellierung) von abstrahierenden Modellen (UML...) ist unumgänglich gerade bei komplexen Software-Architekturen.
- Aber: Eine einzelne Darstellung kann meist die Komplexität und Vielschichtigkeit einer Architektur nicht vermitteln
- Notwendig: Modellierung von verschiedenen Sichten
 - Eine Sicht zeigt das Software-System aus verschiedenen Perspektiven.
 - Abstraktion von Details pro Sicht, Konzentration auf bestimmte Aspekte
 - Sichten können Abhängigkeiten oder Wechselwirkungen haben

Das 4-Sichten-Modell von Starke



(Starke, 2015)



Inhalt:

- Modellierung des Umfelds eines Systems sowie dessen Zusammenhang mit der Umwelt
- Abstrakte Darstellung des Zielsystems als **Blackbox** zu den Nachbarsystemen (keine interne Struktur)
- Darstellung der wichtigsten Schnittstellen (wenn nötig)
- Wichtige Stakeholder des Systems und deren Interaktionen

Zielgruppe:

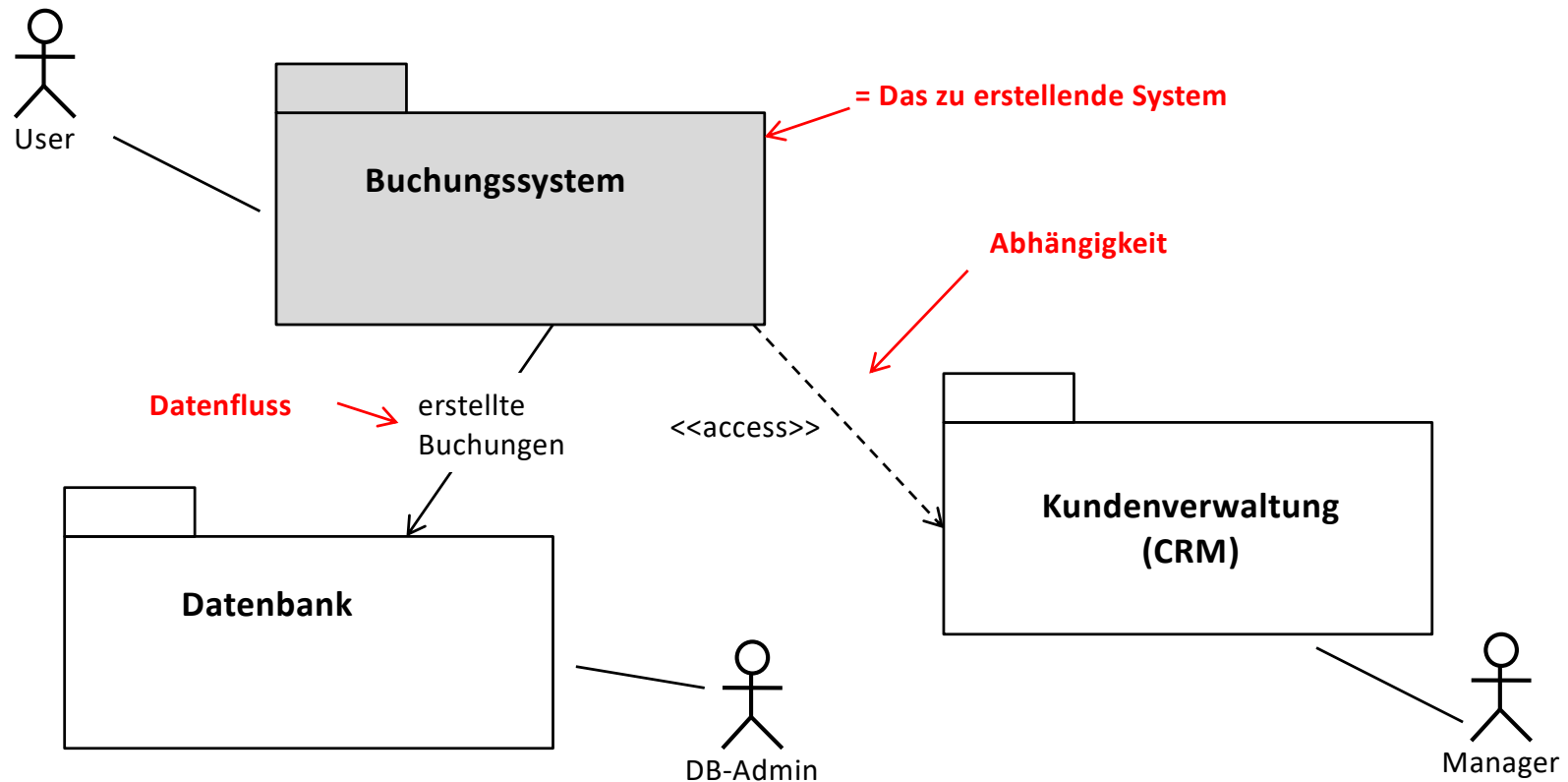
- Sinnvoll für alle Projektbeteiligte als Orientierung
- Auftraggeber oder Kunden, um System-Vision zu verkaufen, erste Validierung hinsichtlich Anforderungen

Darstellung:

- UML **Paketdiagramme** (Rupp, 2012)



- **Paketdiagramme** geben einen Überblick über die Zerlegung des Gesamtsystems in Pakete oder Teilsysteme
- Logische Zusammenfassung von Systemen, meist als Blackbox dargestellt
- Schnittstellen werden nicht berücksichtigt





Inhalt:

- Bausteinsichten zeigen die statischen Strukturen der Architektur-Elemente des Systems, Subsysteme, Komponenten, Klassen und deren Schnittstellen
- Darstellung der Abhängigkeiten zwischen den Elementen
- Darstellung der Elemente als **Whiteboxes** und **Blackboxes**
- Hierarchisierung und Verfeinerung möglich

Zielgruppe:

- Projektleiter und Auftraggeber (Projektüberwachung)
- Zuteilung von Arbeitspaketen (in Form von Architekturbausteinen) an Teams
- Referenz für Software-Entwickler

Darstellung:

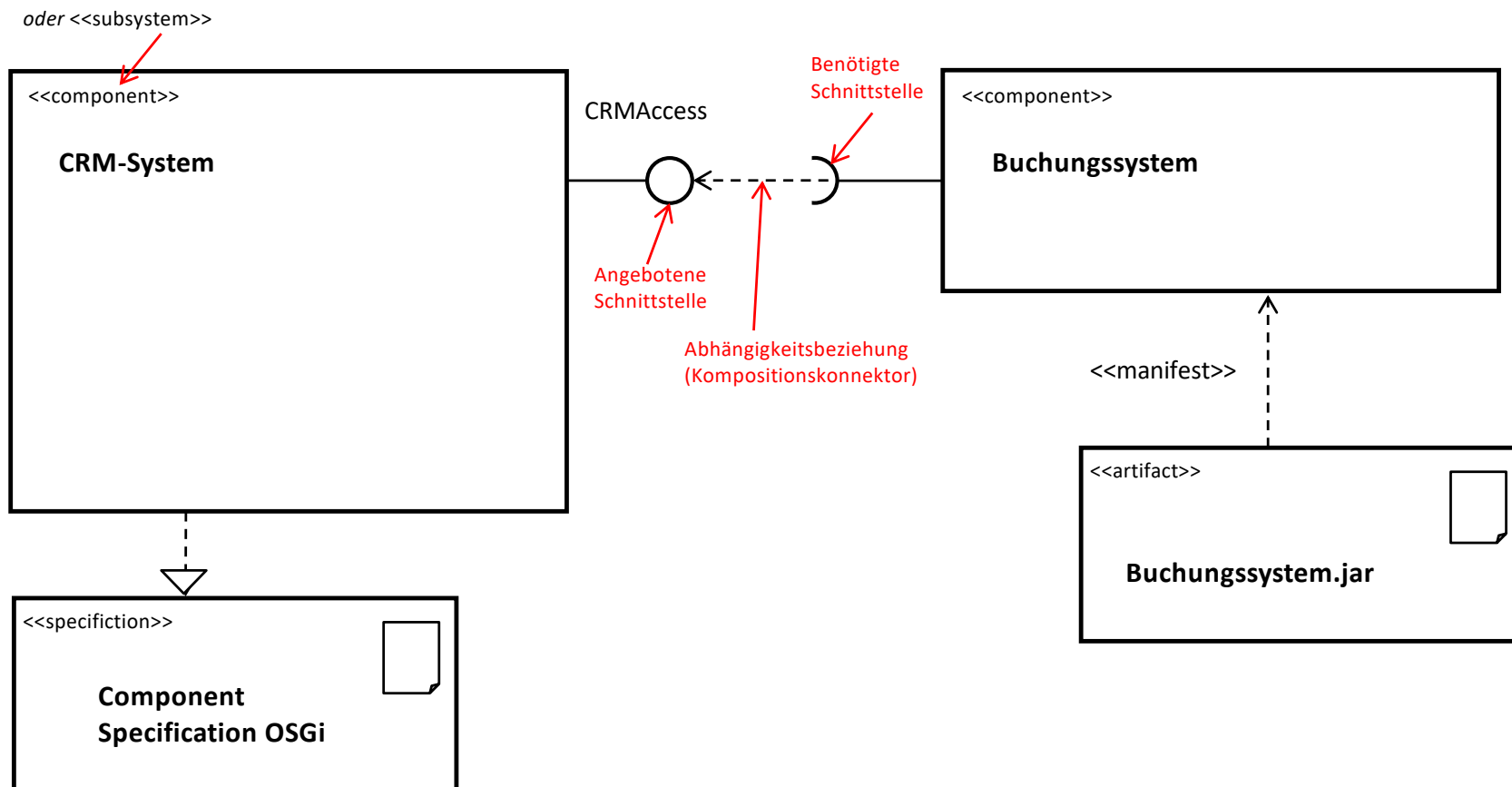
- UML **Komponentendiagramme und Kompositionsstrukturdiagramme**
- UML Klassendiagramme
→ für alle Diagramm-Typen: (Rupp, 2012)!

Dokumentation von Architekturen mit UML

Komponentendiagramme



- Das **Komponentendiagramm** betrachtet Architektur-Elemente (Subsysteme, *einsetzbare* Komponenten), wie sie zur Laufzeit organisiert sind.
- Explizite Modellierung von Schnittstellen (benötigte und angebotene)
- Modellierung der Komponenten als Blackbox

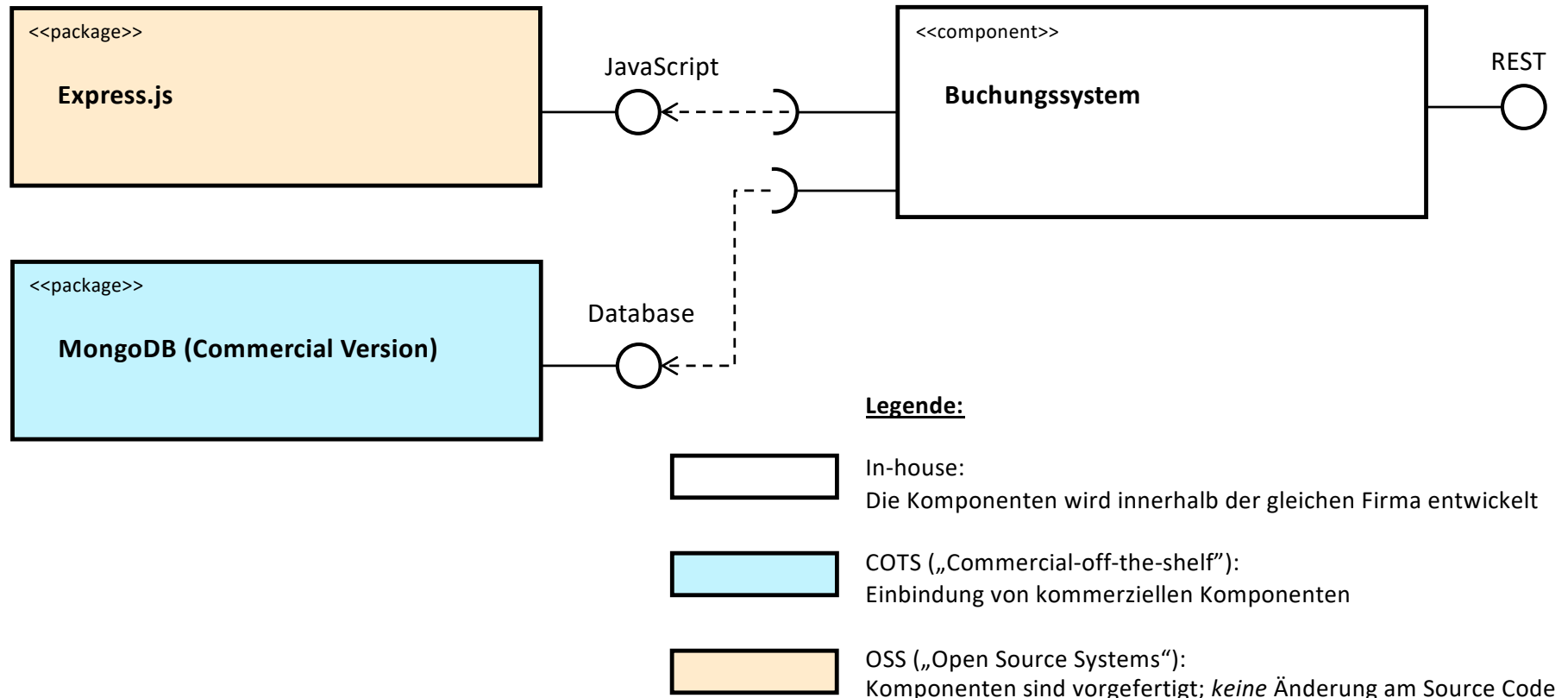


Dokumentation von Architekturen mit UML

Komponentendiagramme



- Das **Komponentendiagramm** betrachtet Architektur-Elemente (Subsysteme, *einsetzbare* Komponenten), wie sie zur Laufzeit organisiert sind.
- Die Verwendung von externen Komponenten von externen Märkten oder Anbietern können durch eine Legende entsprechend markiert werden (kein Standard in UML):

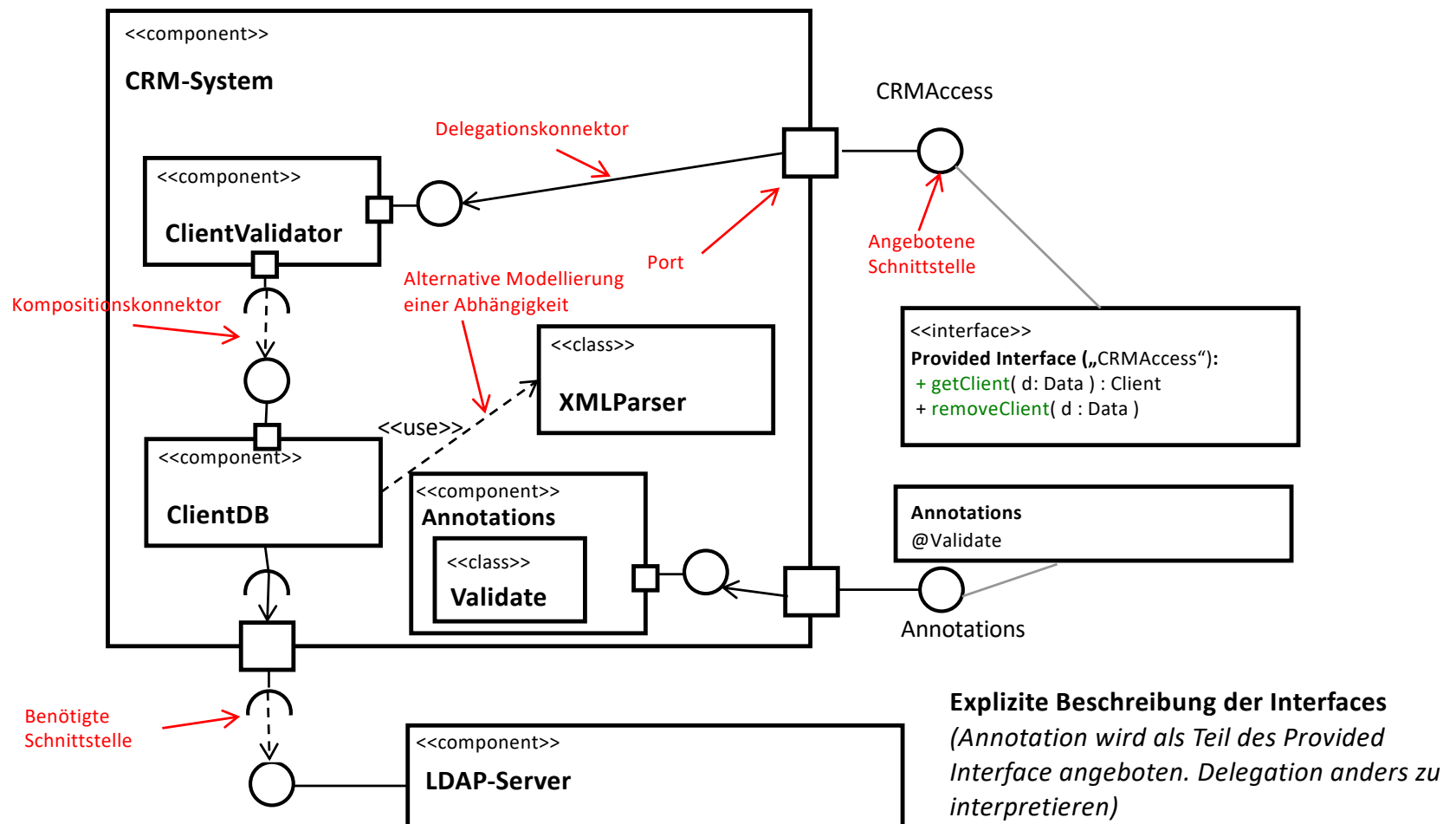


Dokumentation von Architekturen mit UML

Komponentendiagramme / Kompositionsstrukturdiagramme



- Die Struktur einer Komponente (eines Subsystems) kann durch Kompositionsstrukturdiagramme weiter modelliert werden (Whitebox):
 - Ports:** Mittler zwischen externen Verhalten der Komponente und interner Realisierung

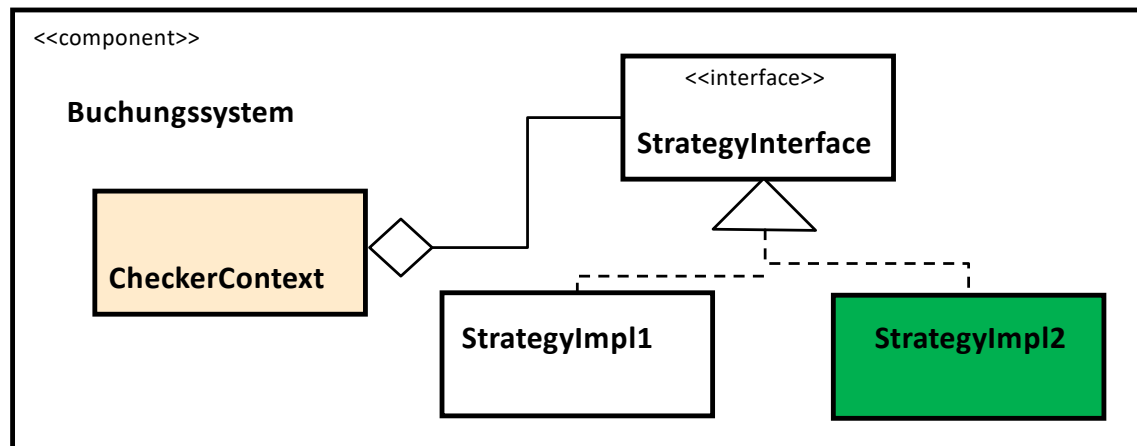


Dokumentation von Architekturen mit UML

Komponentendiagramme



- Das **Komponentendiagramm** betrachtet Architektur-Elemente (Subsysteme, *einsetzbare* Komponenten), wie sie zur Laufzeit organisiert sind.
- Die internen Strukturen können auch mit (internen) Klassen-Diagrammen ergänzt werden. Auch hier kann eine Legende verwendet werden, um interne Weiterentwicklungen zu charakterisieren (Quelle: Projekt ProSyWis, (Veit, 2015))



Legende:



Erweiterung (z.B. neue Unterklasse)

Anpassung (am Code)



Inhalt:

- Die Verteilungssicht beschreibt die Ablaufumgebungen des Systems in Form von **Hardware-Komponenten** (Server, Netzwerke, Firewalls) mit den beteiligten Protokollen.
- Darstellung auch von Laufzeitumgebungen (z.B. Container oder Application Server)
- Modellierung auch, welche Komponenten auf welche Umgebungen deployed sind

Zielgruppe:

- Betreiber des Zielsystems, Entwicklungsteam als Orientierung, Vorstellung ans Projektmanagement

Darstellung:

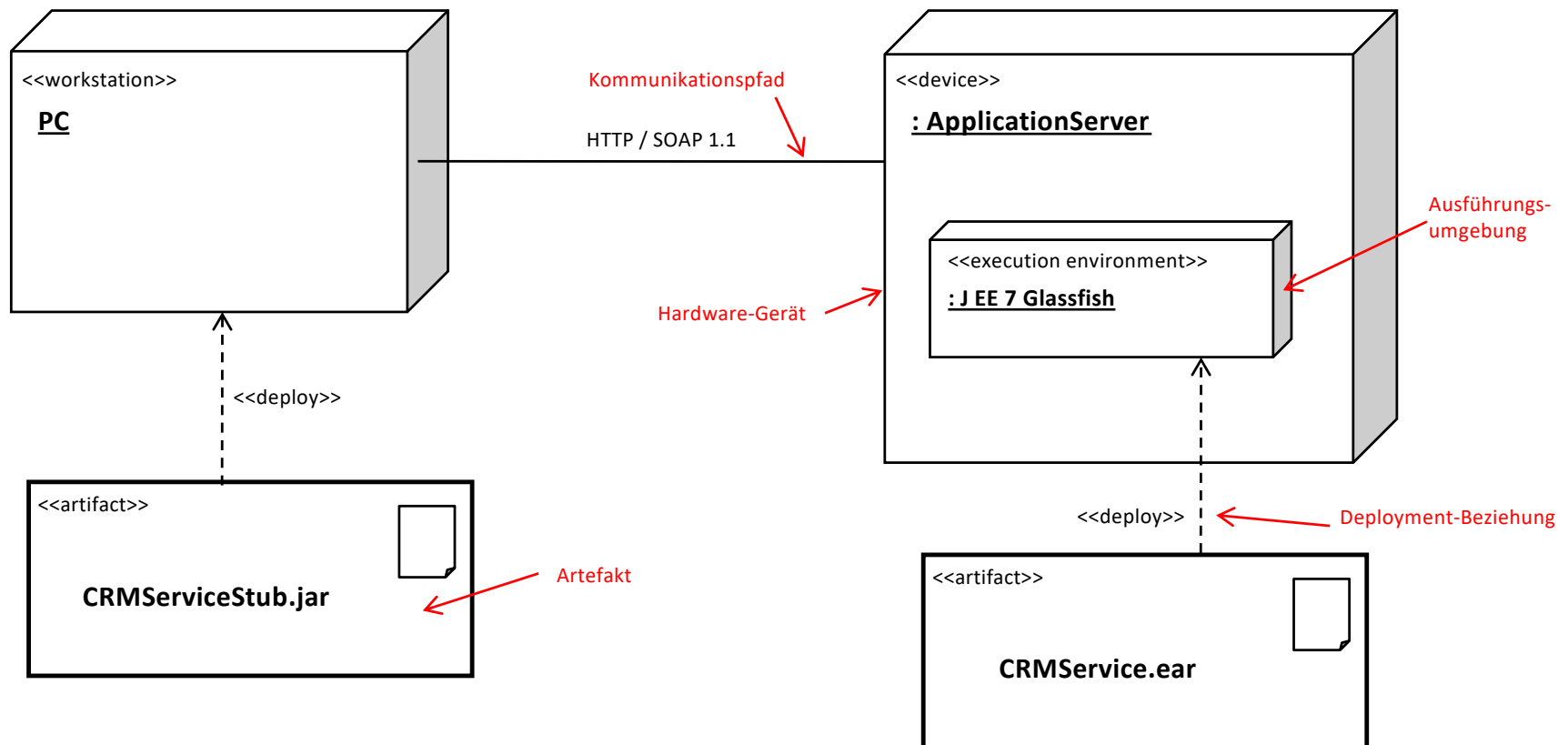
- UML Verteilungsdiagramme

Dokumentation von Architekturen mit UML

Verteilungsdiagramme



- Das **Verteilungsdiagramm** betrachtet die Hardware-Topologie sowie die Verteilung der Software des Systems auf die Hardware

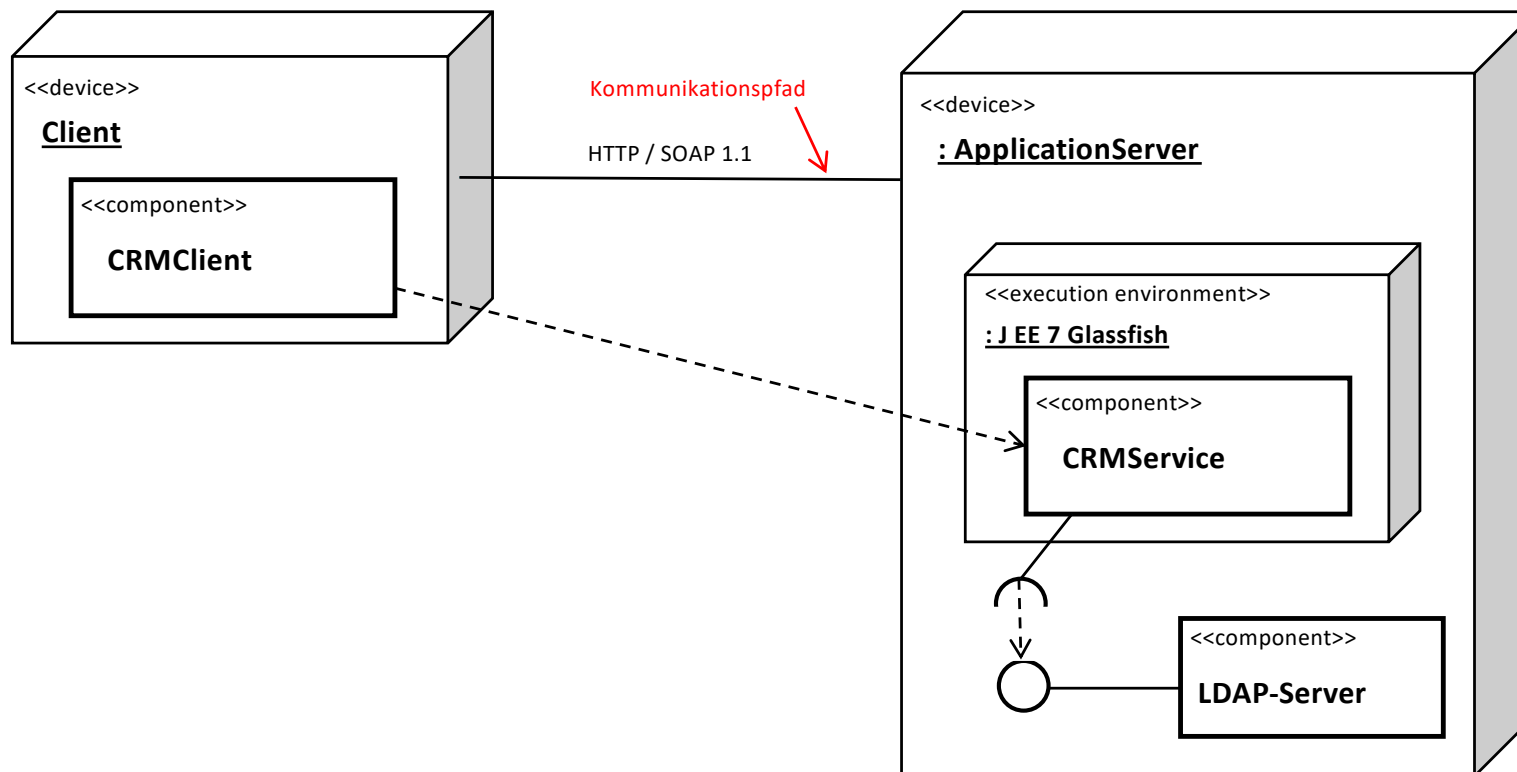


Dokumentation von Architekturen mit UML

Verteilungsdiagramme



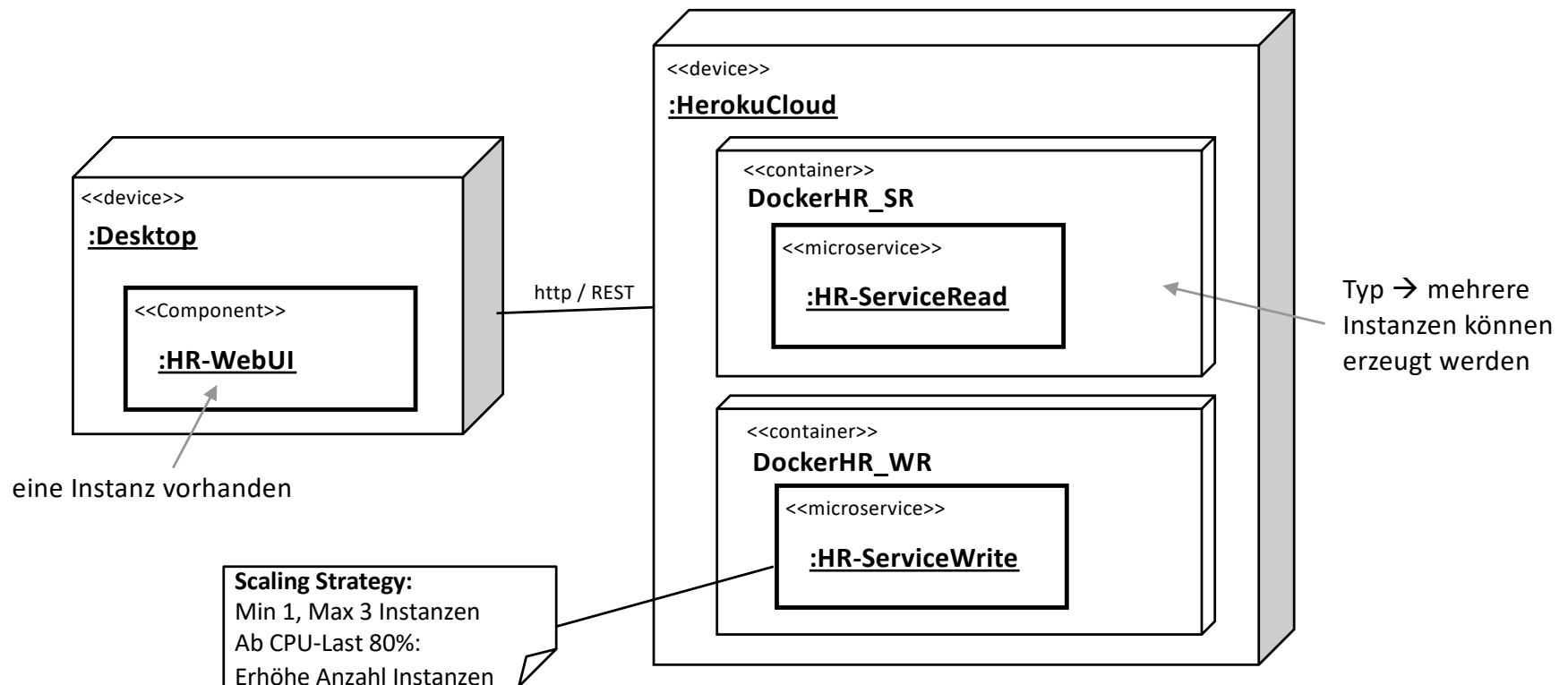
- Das Verteilungsdiagramm und das Komponentendiagramm werden häufig vermischt zu einer integrierten Darstellung:



Modellierung von Microservices und Container (z.B. Docker)



- ♦ **Container** wie z.B. Docker können mittels UML-Komponenten sowie durch einen eigenen Stereotyp modelliert werden.
- ♦ Auch **Microservices** können durch einen Stereotyp modelliert werden
- ♦ Die Möglichkeit, dass die Anzahl von Instanzen von Containern (oder Microservices) durch die Ausführungsumgebung erhöht wird (→ horizontale Skalierung, Auto-Scaling) wird durch die „Typ / Instanz“-Syntax angedeutet.



4-Sichten-Modell: Fazit bis hierin:



- Das 4-Sichten-Modell bietet eine Reihe von verschiedenen Sichten zur Modellierung von Software-Architekturen
- Zentrale Sicht: Baustein-Sicht:
 - Nimmt ca. 60% der Architekturmodellierung in Anspruch
 - Guter Start, wenn ein vorhandenes System verändert werden soll und somit Teile der Baustein-Sicht vorgegeben sind.
 - Optimales Dokument für Abstimmung aller Stakeholder
- Empfehlung: Pragmatischer Einsatz: nicht zu viel modellieren und dokumentieren, auf bedarfsgerechten Einsatz achten
- Was fehlt beim 4-Sichten-Modell: eine **Methodik**, **wie** man Anforderungen auf eine Architektur abbildet (→ siehe nächster Abschnitt!)



Kapitel 3: Modellierung und Entwicklung von komponentenbasierten Software-Architekturen

1	Vorgehensmodelle zur Entwicklung von komponentenbasierten Architekturen und zur Bewertung von Komponenten	✓
2	Semi-formale Modellierung von Architekturen: Das 4 Sichten-Modell	✓
3	Weitere Formen zur Dokumentation und formalen Modellierung von Software-Architekturen (arc42, ADL)	
4	Referenz-Architekturen für komponentenbasierte Architekturen	
5	Zusammenfassung und Ausblick	



- Modellbasierte Modellierung einer Software-Architektur
 - Basierend auf graph-basierten Modellierungselementen
 - **Semi-formal** (Semantik oft unpräzise oder mehrdeutig)
 - Dienen zur Kommunikation zwischen Entwicklern und Kunden
 - Aktueller Industrie-Standard: Unified Modeling Language UML 2.5
- Architecture Description Language (ADL)
 - Textuelle Spezifikation von Architekturen (insbesondere der Interaktionsbeziehungen)
 - **Formal** (Semantik klar formalisiert)
 - Weniger zur Kommunikation, dient der direkten Umsetzung sowie Analyse
 - Meist akademische Ansätze (z.B. **ACME**, Darwin, C2, PeerCAT)
 - Vorläufer von heutigen Sprachen wie BPEL oder **(Docker) Compose**
- Textuelle Architektur-Templates
 - Freisprachliche Beschreibung der Architekturelemente
 - **Keine formalen Vorgaben** (keine Semantik vorgegeben)
 - Wenige Standards vorhanden (in Deutschland populär: arc42)



Prof. Jeff Kramer
(Entwickler von Darwin)
(„YES!”)



- Leichtgewichtiges Framework zur Beschreibung der wesentlichen Aspekte auf eine Architektur (gemäß 4-Sichten Modell)
- Resultierendes Dokument sollte alle architekturelevanten Informationen enthalten:
 - Architekturziele
 - Qualitätsanforderungen
 - Sichten
 - Entwurfsentscheidungen
 - Muster usw.
- Quelle: <http://www.arc42.de>
- Umfang kann durchaus komplex sein (> 20-100 Seiten oder *mehr?*). Kompaktere Übersichten können nach (Starke, 2015) abgeleitet werden:
 - Übersichtspräsentation der Architektur (vgl. Semesterprojekt)
 - Architekturapete (Single-Page View für Management)

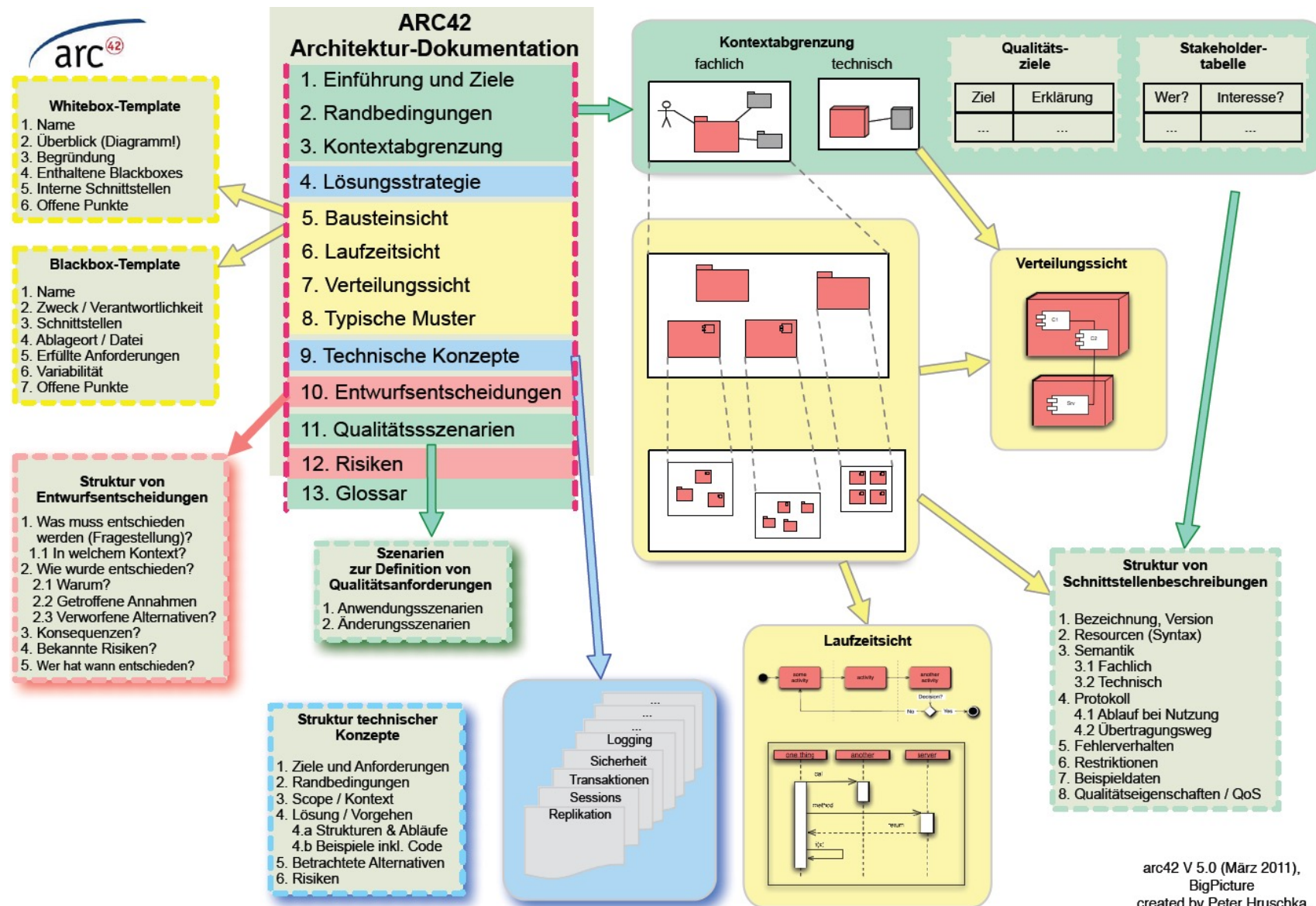


- Web-basiertes Tool zur Analyse und Dokumentation einer Architekturbeschreibung basierend auf arc42 (2023- heute)
- Erstes Release: <https://github.com/aldaGit/arc42doc>
- Aktuelle Arbeiten: Integration von MCP und LLM zur automatisierten Generierung einer Dokumentation (MA Niederprüm) und für das gezielte Onboarding von neuen Projektmitgliedern (MA Lück)

The screenshot shows the arc42doc web application. The sidebar on the left contains a list of navigation items: 1. Einführung und Ziele, Aufgabenstellung, Qualitätsziele (selected), Stakeholder, Nachhaltigkeitsziele, 2. Randbedingungen, 3. Kontextabgrenzung, 4. Lösungsstrategie, 5. Bausteinsicht, Bausteinsicht-Diagramm, Bausteinsicht-Beschreib..., 6. Laufzeitsicht, 7. Verteilungssicht, 8. Konzepte, 9. Entwurfsentscheidungen, 10. Qualitätsszenarien. The main content area is titled 'Einführung und Ziele > Qualitätsziele >' and contains a table with the following data:

ID	Qualitätsziel	Motivation	Kategorie
116	Das System muss in der Lage sein, auch bei hoher Benutzerlast und großen Datenmengen schnelle Antwortzeiten zu gewährleisten.	Um sicherzustellen, dass die Benutzer effizient arbeiten können, muss das System schnell und reaktionsfähig bleiben, unabhängig von der Anzahl der gleichzeitigen Benutzer oder der Menge der verwalteten Daten.	Effizienz
117	Schutz der Benutzerdaten und Sicherstellung der Vertraulichkeit, Integrität und Verfügbarkeit der Informationen.	Da das System sensible Projekt- und Benutzerdaten verwaltet, ist es entscheidend, dass diese Daten vor unbefugtem Zugriff und Manipulation geschützt werden.	Sicherheit
118	Das Tool muss eine intuitive und leicht verständliche Benutzeroberfläche bieten.	Eine einfache Bedienbarkeit erhöht die Akzeptanz des Tools bei den Benutzern und reduziert den Schulungsaufwand.	Benutzbarkeit
119	Das System muss eine hohe Verfügbarkeit und Fehlertoleranz bieten.	Unterbrechungen im Betrieb können zu Produktivitätsverlusten führen, daher muss das System möglichst ausfallsicher sein.	Zuverlässigkeit
120	Der Code und die Systemarchitektur sollen	Um schnelle Anpassungen und Erweiterungen des Systems zu ermöglichen, müssen der	Wartbarkeit

Big Picture zu arc42



arc42 V 5.0 (März 2011),
BigPicture
created by Peter Hruschka
& Gernot Starke



- ACME ist eine ADL zur Beschreibung und Analyse von komponentenbasierten Architekturen.
- Etablierung einer eindeutigen Ontologie zur Bestimmung einer konzeptuellen Basis (Component, Connector, System, Properties, Constraints, Styles)
- Anbindung an verschiedenen Programmiersprachen (Java, C++)
- Tools zur Entwicklung (z.B. Eclipse)
- Quelle:
 - http://www.cs.cmu.edu/~acme/docs/language_overview.html
 - Garlan, David; Monroe, Robert; and Wile, David, "Acme: Architectural Description of Component-Based Systems" (2000). *Computer Science Department. Paper 693.*

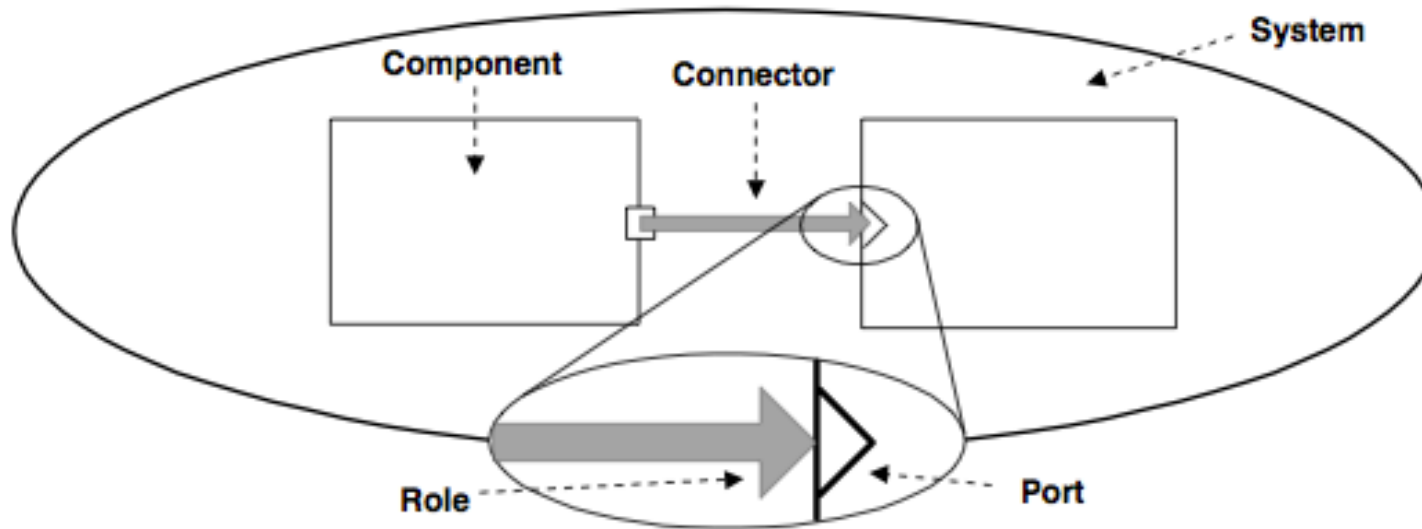


Fig. 3.2. Elements of an Acme Description.

```
System simple_cs = {  
  Component client = { Port sendRequest }  
  Component server = { Port receiveRequest }  
  Connector rpc = { Roles {caller, callee} }  
  Attachments : {  
    client.sendRequest to rpc.caller ;  
    server.receiveRequest to rpc.callee }  
}
```

Quelle: Garlan, David; Monroe, Robert; and Wile, David, "Acme: Architectural Description of Component-Based Systems" (2000).
Computer Science Department. Paper 693.

Docker Compose (Compose specification)



```
version: '3.9'
services:

  app:
    image: python:3.12
    container_name: flask_app

    ports:
      - "5000:5000"

    depends_on:
      - db

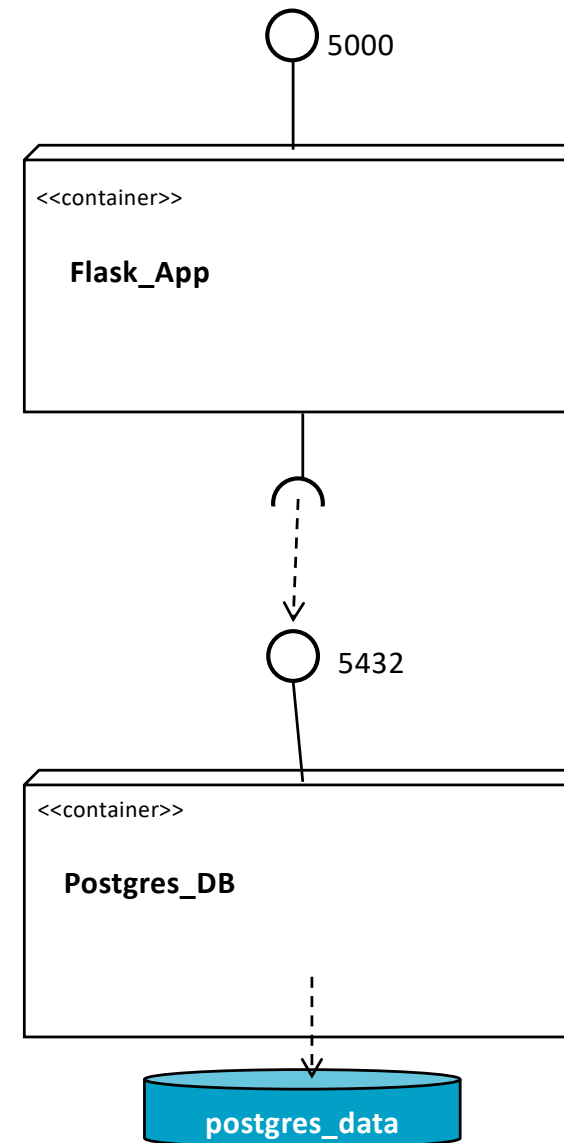
  db:
    image: postgres:16
    container_name: postgres_db

    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: geheim
      POSTGRES_DB: beispiel

    ports:
      - "5432:5432"

    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```



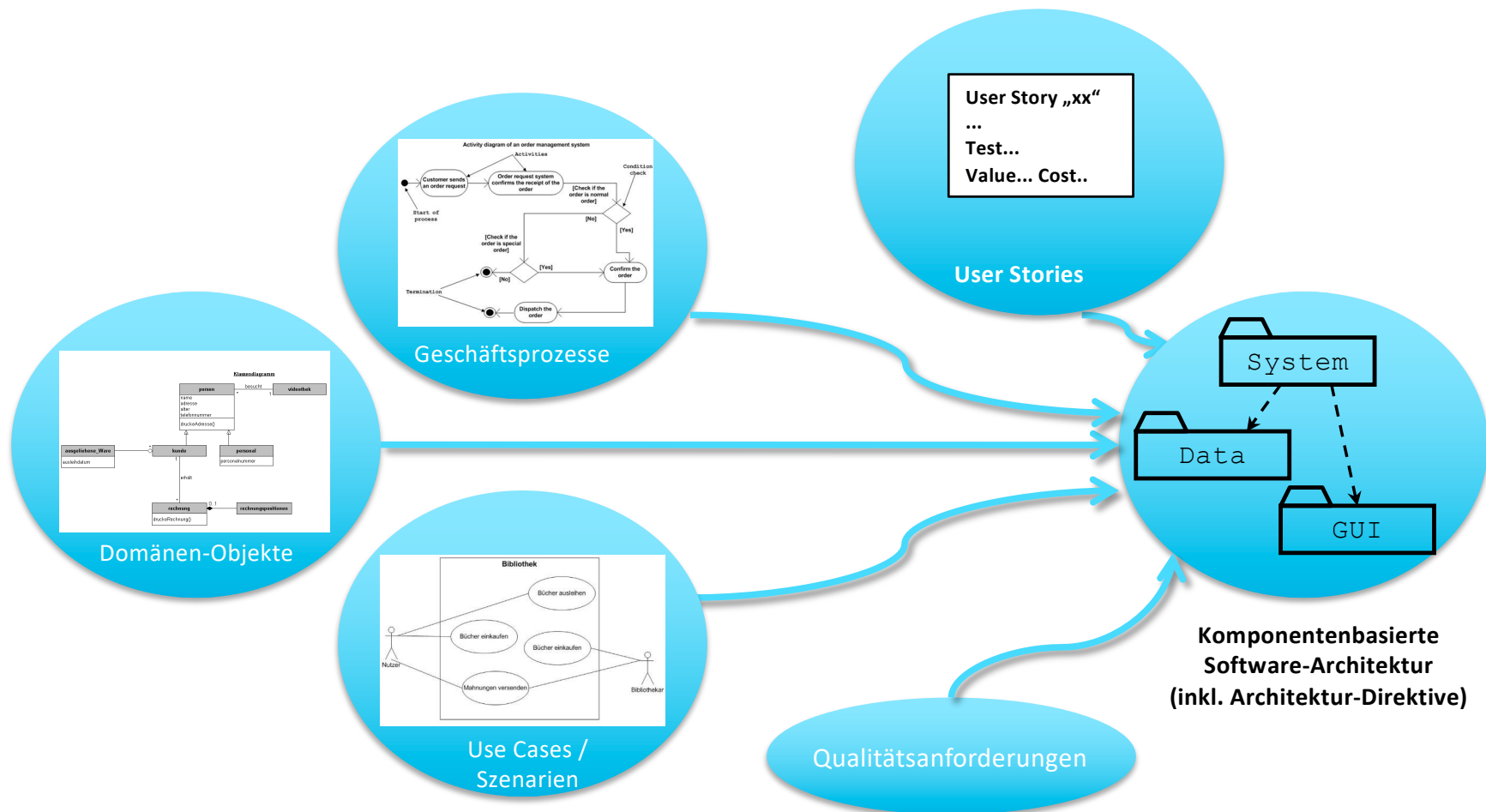
Quelle Specification: <https://docs.docker.com/reference/compose-file/>



Kapitel 3: Modellierung und Entwicklung von komponentenbasierten Software-Architekturen

1	Vorgehensmodelle zur Entwicklung von komponentenbasierten Architekturen und zur Bewertung von Komponenten	✓
2	Semi-formale Modellierung von Architekturen: Das 4 Sichten-Modell	✓
3	Weitere Formen zur Dokumentation und formalen Modellierung von Software-Architekturen (arc42, ADL)	✓
4	Referenz-Architekturen für komponentenbasierte Architekturen	
5	Zusammenfassung und Ausblick	

- Die Aufgabe der Phase „System Design“ ist die Entwicklung einer Software-Architektur basierend auf den ermittelten Anforderungen des Ziel-Systems

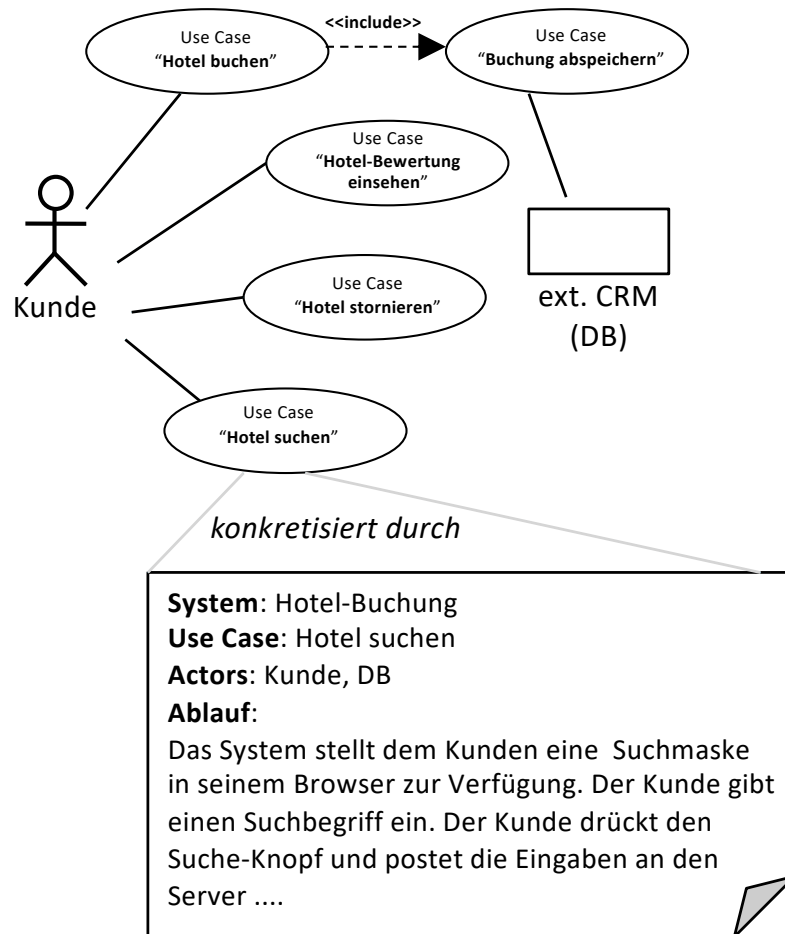




- Es werden drei Methoden zur Entwicklung von Komponenten-Architekturen auf Basis von Artefakten aus der Anforderungsanalyse betrachtet.
- Klassischer Ansatz durch die Analyse von Anforderungsartefakten, **Abbildung auf eine (objektorientierte) Komponenten-Architektur** (Brügge und Dutoit, 2013)
 - Diskussion Fallstudien Architektur für Java EE und Microservices
 - Darstellung der Software-Architektur auf verschiedenen Levels (Starke, 2020)
- Entwicklung einer Komponenten-Architektur gemäß **Referenz-Architektur** (Oestereich, 2009)
- Weiterer Ansatz: **Domain Driven Design (DDD)** (Evans, 2004)
 - Strukturierung von komplexen Systemen nach Fachlichkeit
 - Theoretischer Ansatz, dadurch (?) lange Zeit ignoriert, allenfalls im akademischen Bereich
 - Renaissance von DDD im Bereich Microservices (→ Kapitel 2)



- Ansatz: aus (graphischen und textuellen) Use Case Beschreibungen, Szenarien usw. wird während der Analyse ein **objektorientiertes Modell** erstellt



Textuelle Beschreibung eines Use Case



- Anwendung der Analyse-Methode nach Abbott (Abbott, 1983):
 - Substantive = Klassen, Verben = Methoden (u.v.a.)

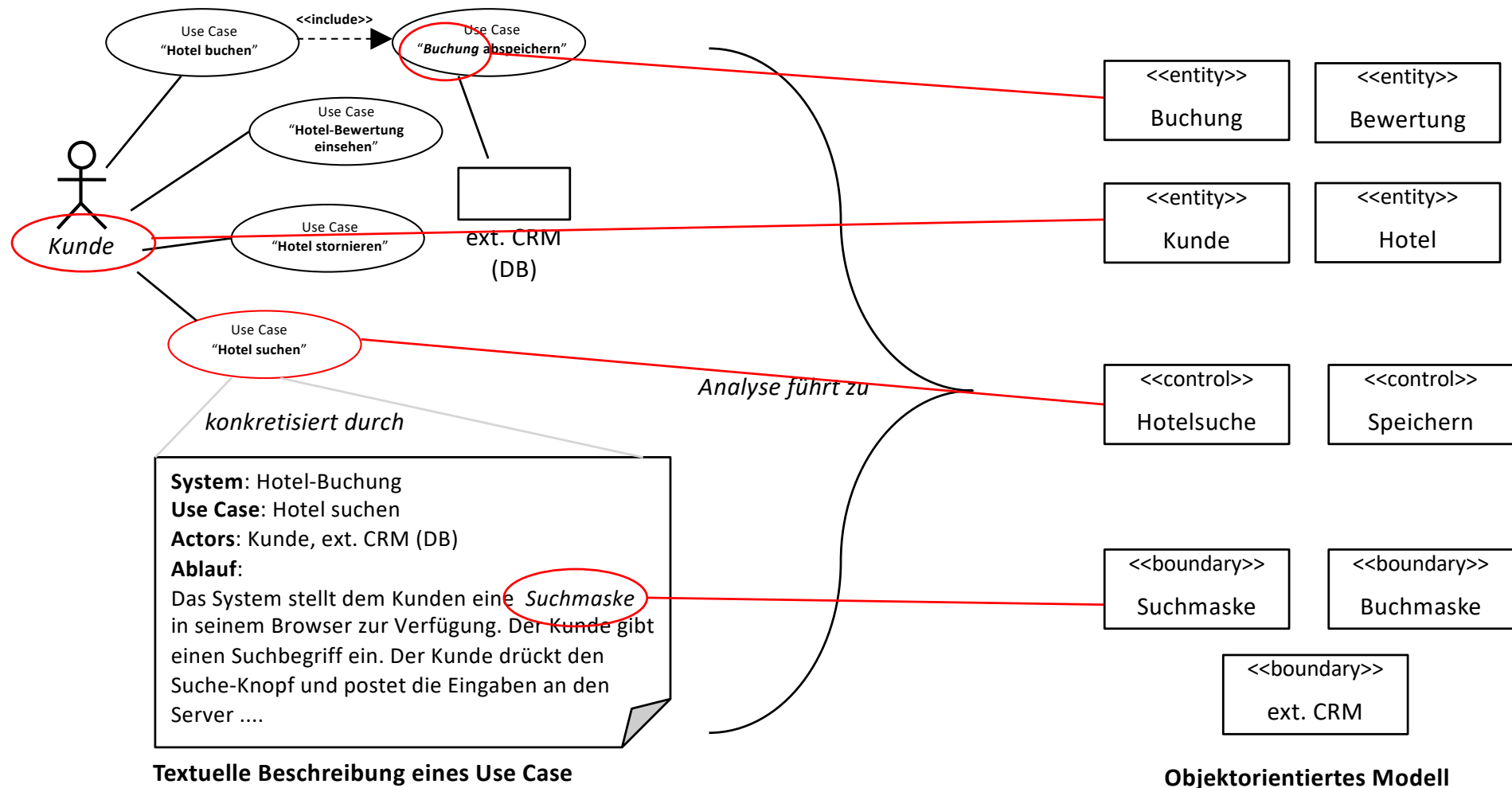
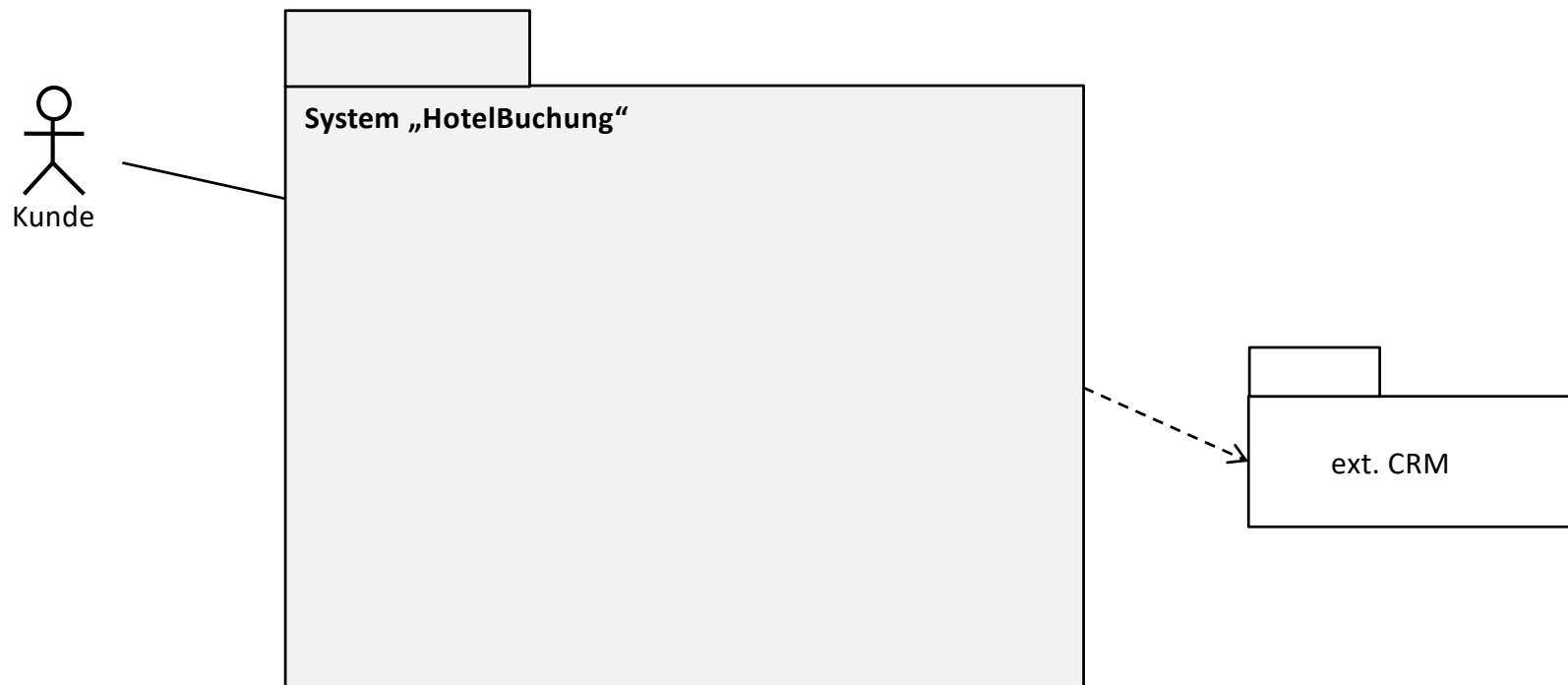


Abbildung typisiertes Objektmodell auf eine Architektur



- Das typisierte Objektmodell ist auf eine Architektur abzubilden
- Vorgehen: schrittweise (iterative) **Dekomposition** der Architektur, Abbildung der Objekte auf konkrete Architektur-Elemente
- Darstellung: Kontext-Sicht, **Blackbox** (interne Details werden nicht modelliert)

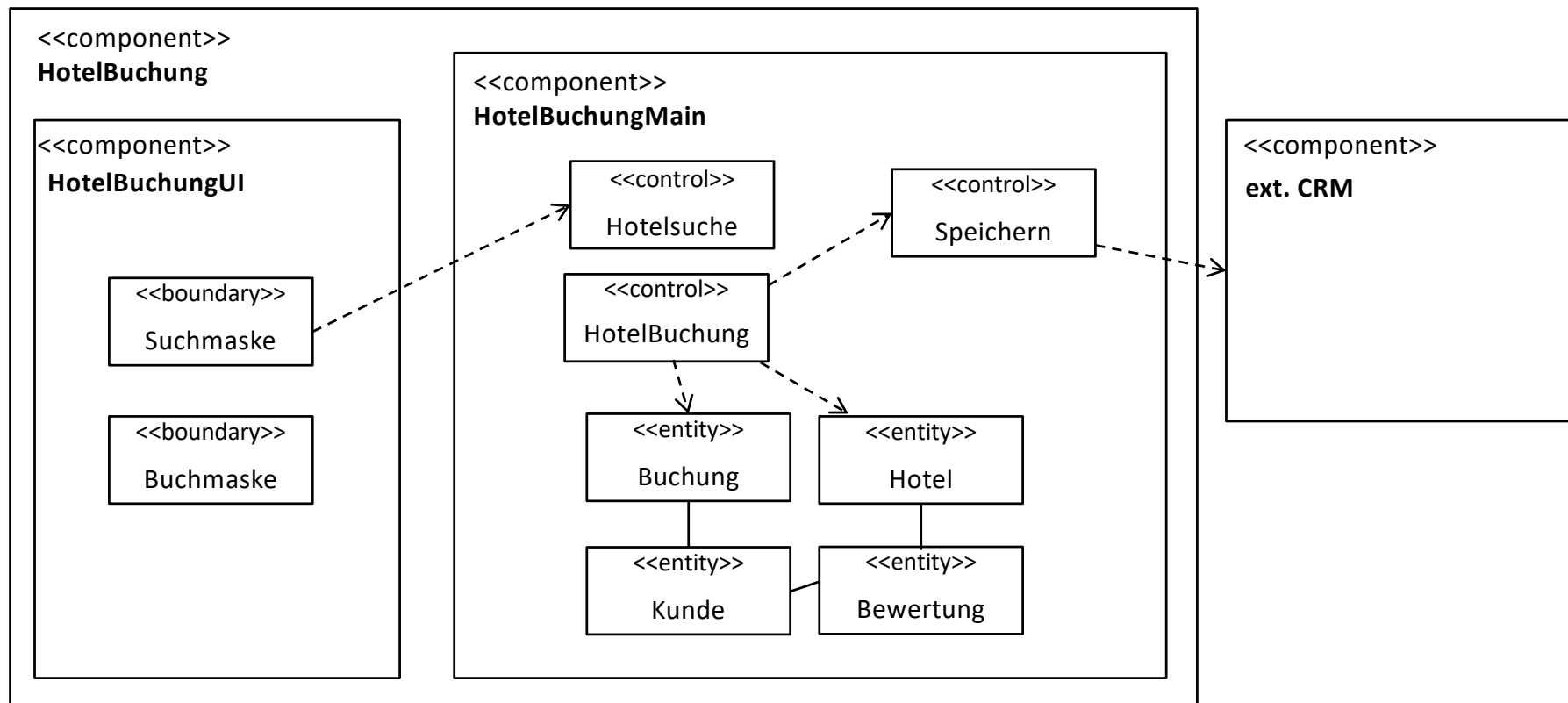


Kontext-Sicht der High-Level-Architektur
(ein Architektur-Element = das GesamtSystem, Level 0)

Abbildung typisiertes Objektmodell auf eine Architektur



- Das typisierte Objektmodell ist auf eine Architektur abzubilden
- Vorgehen: schrittweise (iterative) **Dekomposition** der Architektur, Abbildung der Objekte auf konkrete Architektur-Elemente und **ihre Abhängigkeiten**



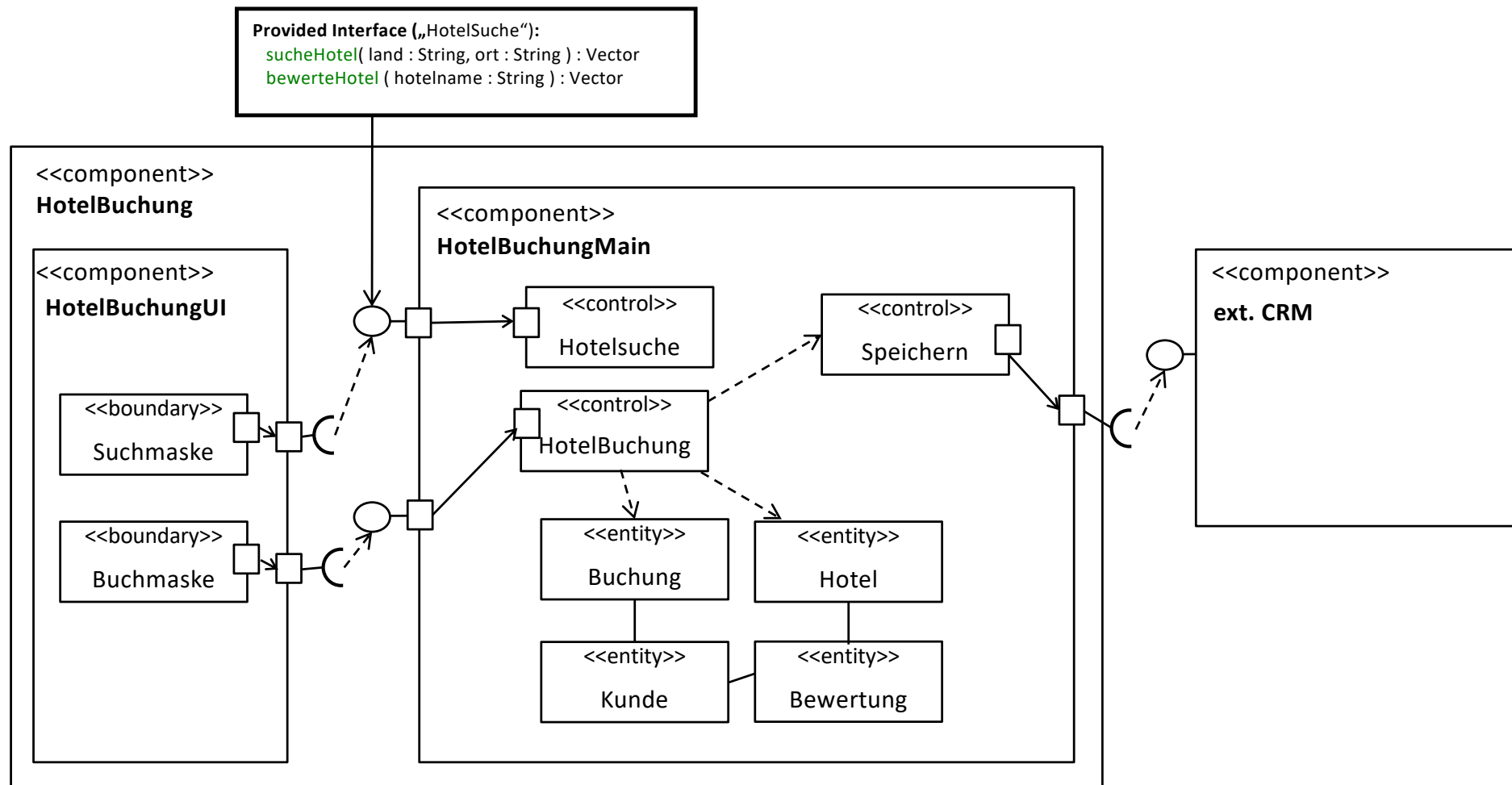
Baustein-Sicht der Architektur, Level 2, nächste Iteration*

*Weitere Aufteilungsschritte von Level 0 ausgehend siehe Anhang

Abbildung typisiertes Objektmodell auf eine Architektur

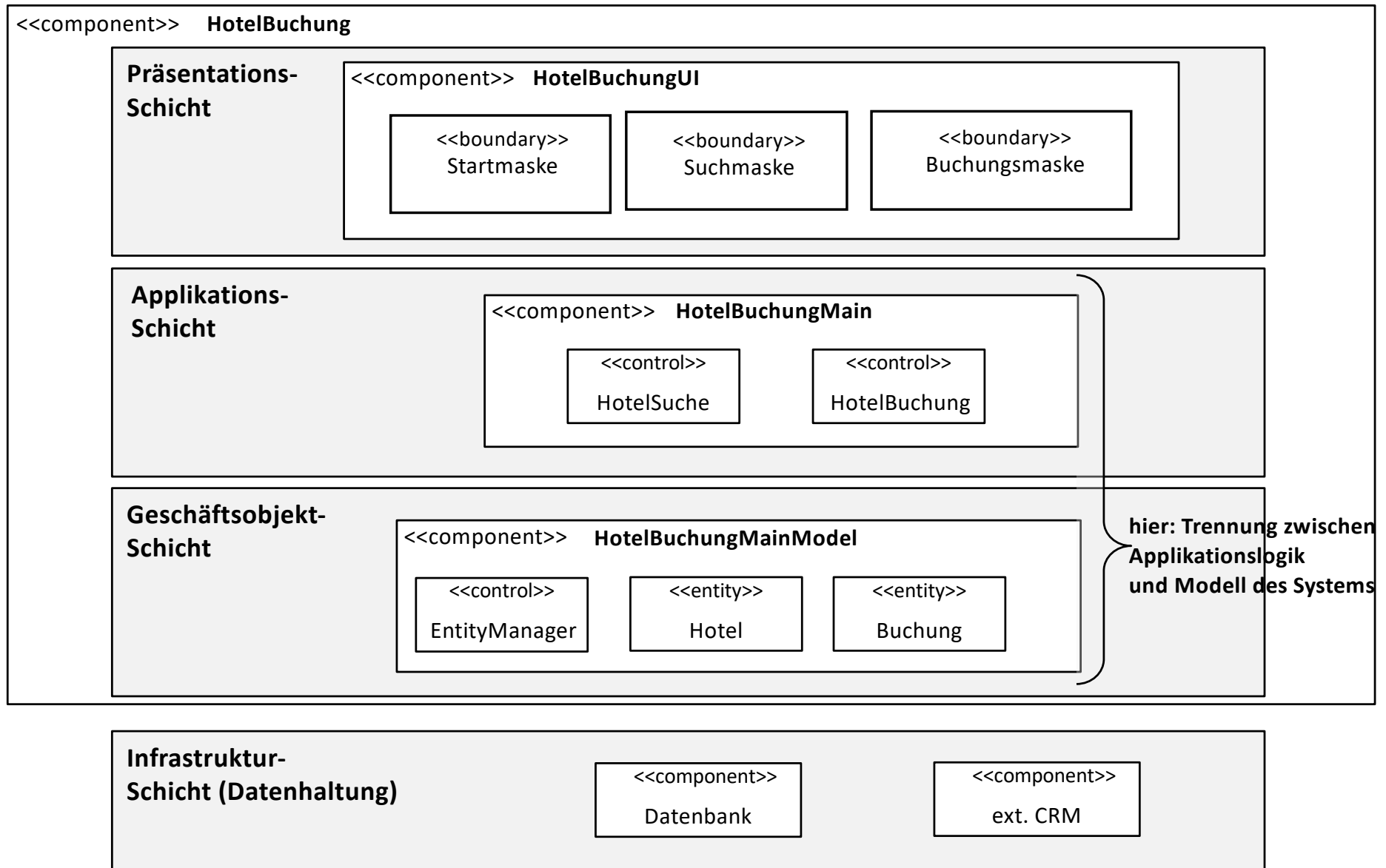


- Identifikation der öffentlichen Schnittstellen der Subsysteme
- Spätestens hier: **Component Assessment** (Make or Reuse (COTS, OSS ...))

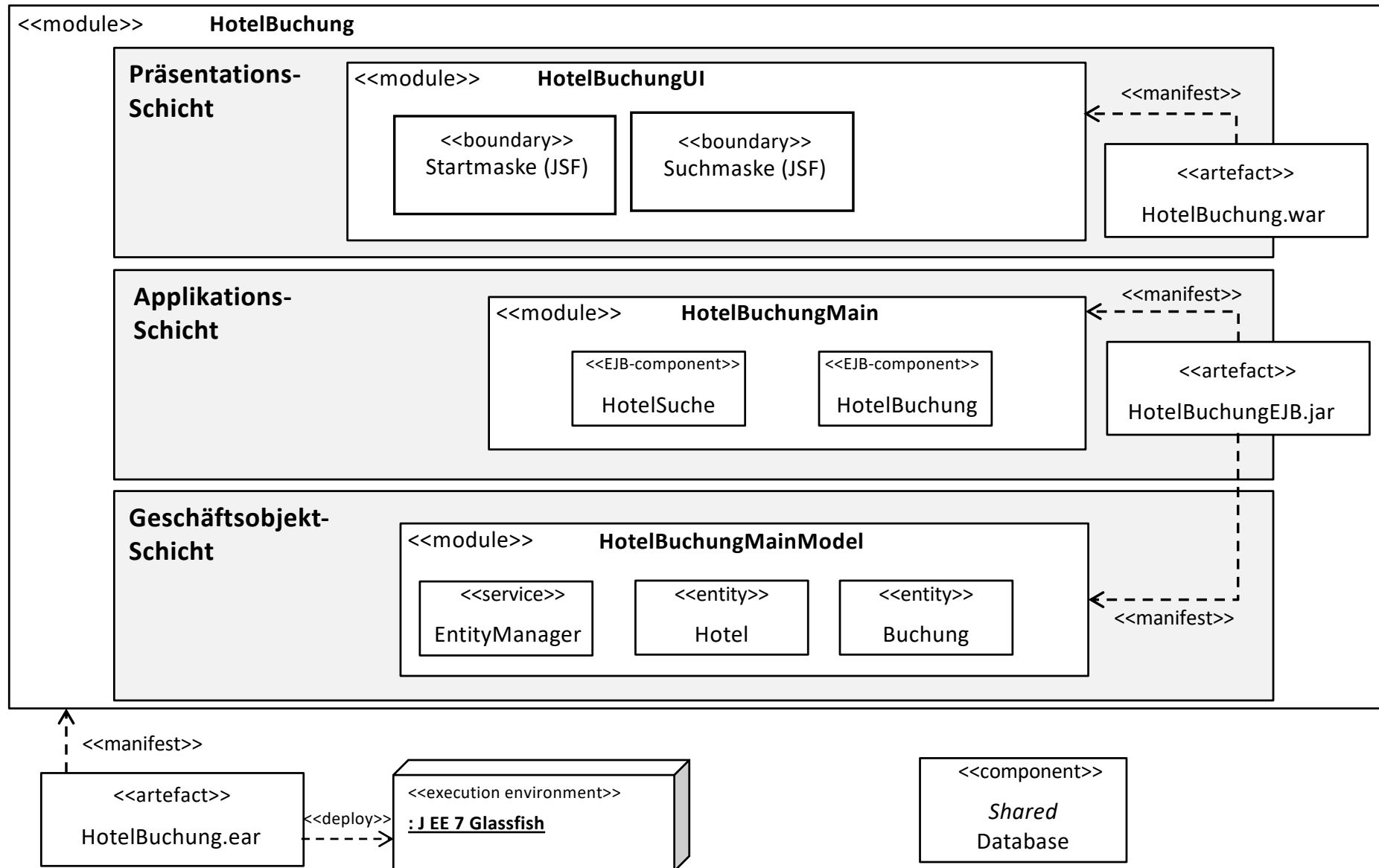


Baustein-Sicht der Architektur, Level 2, nächste Iteration

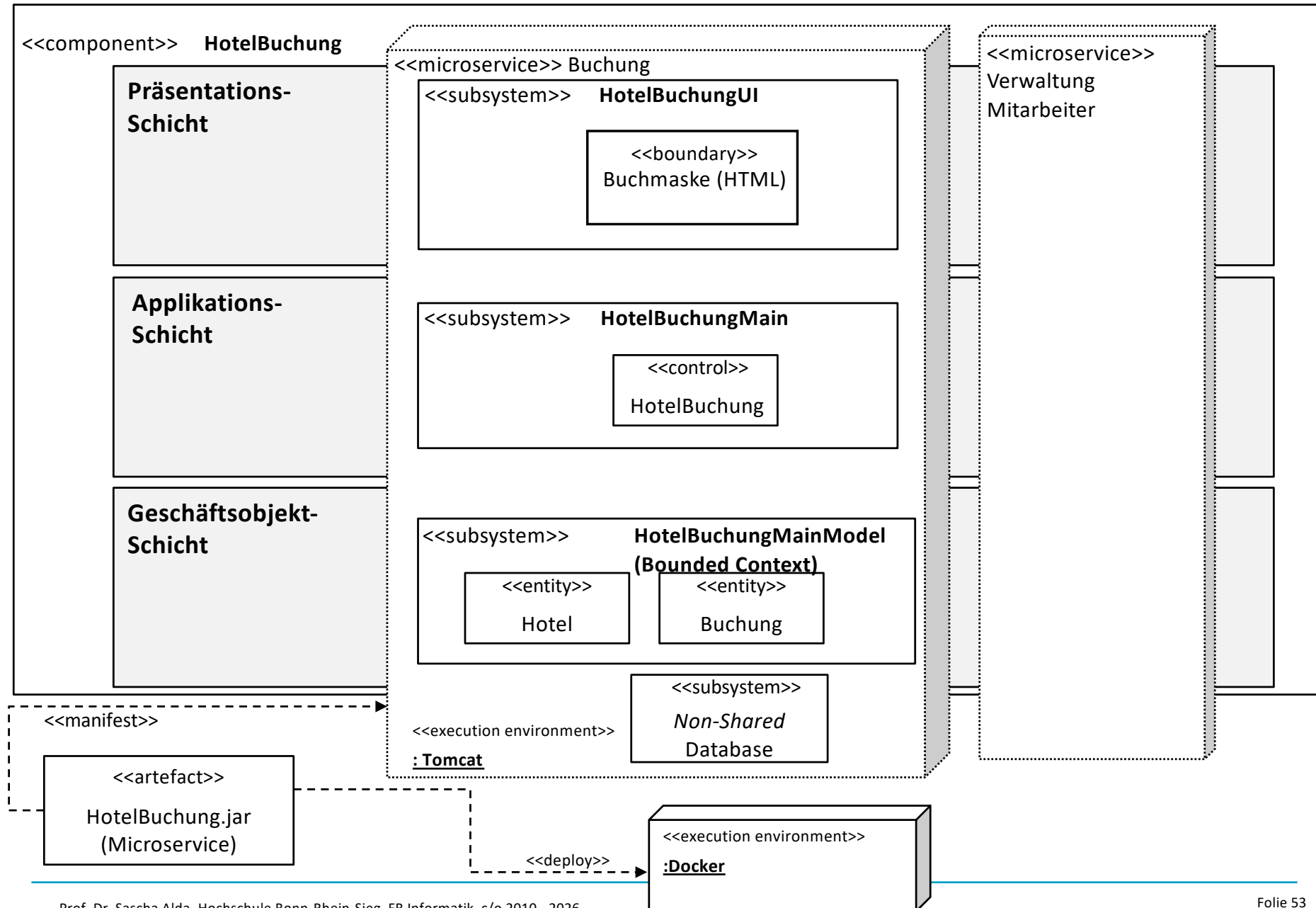
Typischer Aufbau einer Schicht-Architektur



Aufbau einer Schicht-Architektur – Beispiel J EE (Kap. 6)



Aufbau einer Schicht-Architektur – Beispiel Microservice (Kap. 2 / 6)



```

graph LR
    A(( )) --> B[Schichtenmodell definieren]
  
```



Weitere Infos: Siehe Anhang!



Kapitel 3: Modellierung und Entwicklung von komponentenbasierten Software-Architekturen

1	Vorgehensmodelle zur Entwicklung von komponentenbasierten Architekturen und zur Bewertung von Komponenten	✓
2	Semi-formale Modellierung von Architekturen: Das 4 Sichten-Modell	✓
3	Weitere Formen zur Dokumentation und formalen Modellierung von Software-Architekturen (arc42, ADL)	✓
4	Referenz-Architekturen für komponentenbasierte Architekturen	✓
5	Zusammenfassung und Ausblick	



Zusammenfassung

- Die Modellierung einer Software-Architektur sollte aus verschiedenen Sichten erfolgen. Semi-formale Ansätze haben sich dabei etabliert.
- Für die Abbildung von Anforderungen und Rahmenbedingungen auf eine (komponenten-basierte) Software-Architektur gibt es **keine standardisierte Methodik**
 - Anwendung von verschiedenen Methoden und Heuristiken möglich
 - Basiert letztendlich auf den Erfahrungen des Teams / der Software-Entwickler
- Methoden, welche die Meta-Daten aus Änderungshistorien zur Bewertung und Entwicklung von Komponenten verwenden, gewinnen an Bedeutung

Ausblick

- Weitere Muster für die Entwicklung von Software-Architekturen



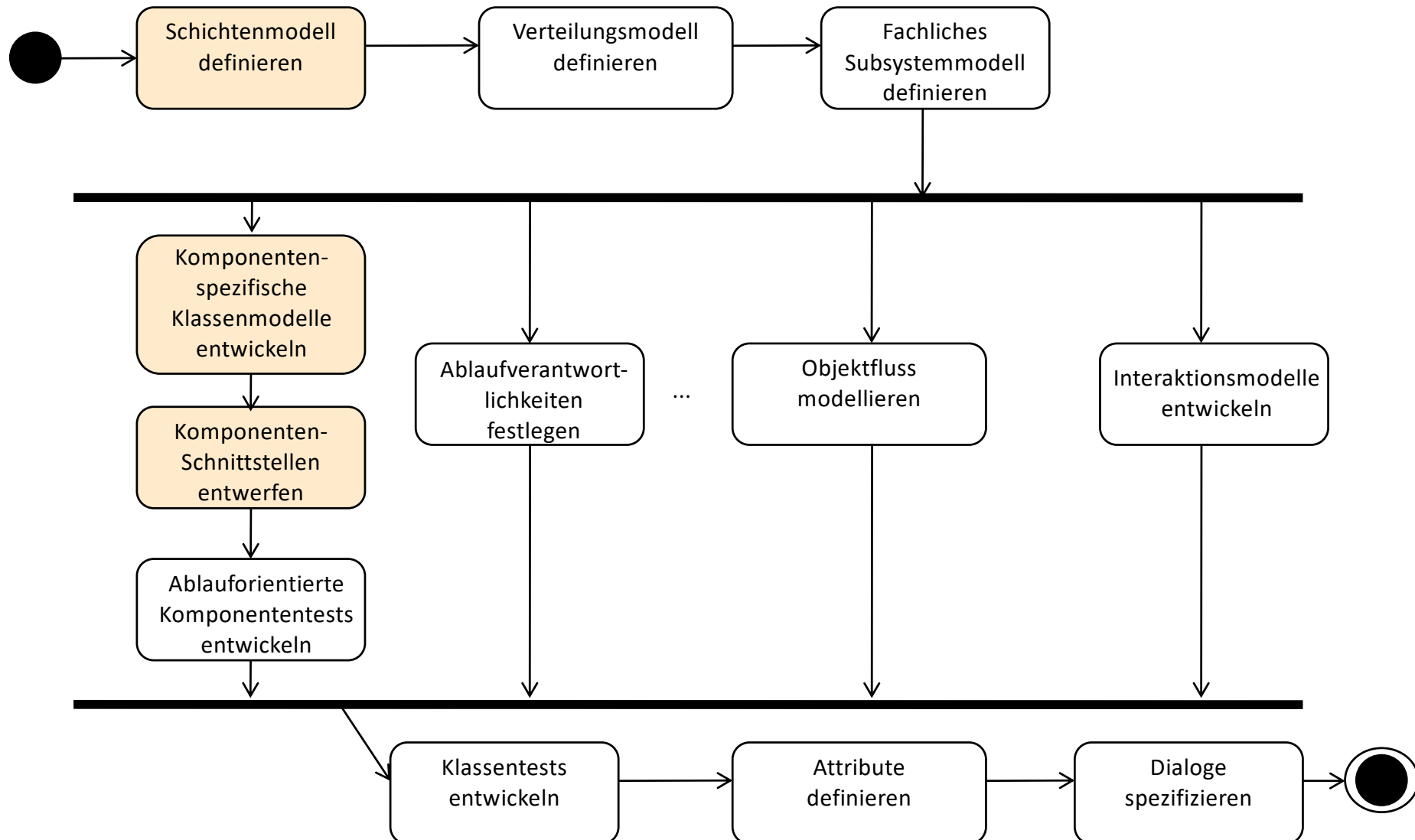
- J. Abbott, Program design by informal English descriptions, Communications of the ACM, v.26 n.11, p.882-894, Nov. 1983
- Evans, Eric: Domain-Driven Design – Tackling Complexity in the Heart of Software. Addison-Wesley, 2004.
- Gamma et al.: Design Patterns, Addison-Wesley, 1996
- Garlan, David; Monroe, Robert; and Wile, David, "Acme: Architectural Description of Component-Based Systems" (2000). Computer Science Department. Paper 693.
- Garret, et al.: The Elements of User Experience, 2003
- Gloger, Boris: Scrum – Produkte zuverlässig und schnell entwickeln, Hanser 2009
- Kruchten, P.: „The 4+1 View Model of Architecture“, in: IEEE Software, Vol. 12, Nr. 6. 1995.
- (Ernst, N., Kazman, R. und Bianco, P.: „Component Comparison, Evaluation, and Selection: A Continuous Approach“, in: Proceeding of the International Conference on Software Architecture Companion (ICSA-C). IEEE Publishing, 2019.



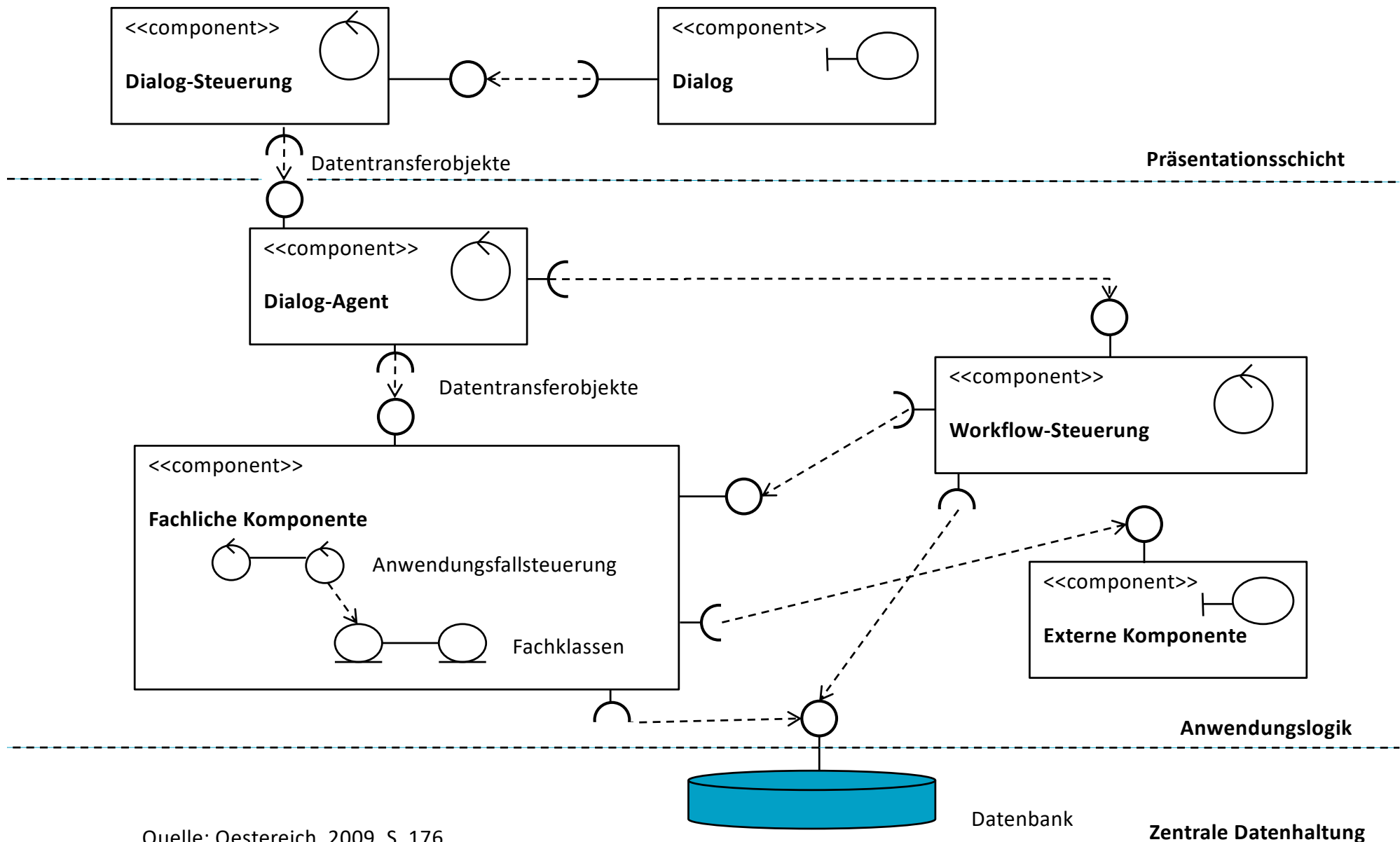
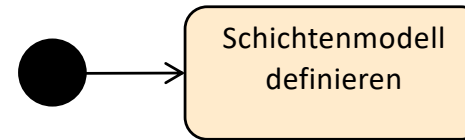
- Oestereich, B. : Analyse und Design mit UML 2.3. Oldenburg Verlag, 2009
- Crnkovic, I.; Chaudron, M.; Larsson, S.: „Component-Based Development Process and Component Lifecycle“, in: International Conference on Software Engineering Advances. 2006
- Petersen, K. Badampudi, D., Crnkovic, I et al.: Choosing Component Origins for Software Intensive Systems: In-house, COTS, OSS or Outsourcing? -- A Case Survey. IEEE Transactions on Software Engineering (Volume: 44, Issue 3). March 2018
- Rupp, C.: UML2 Glasklar. Hanser Verlag, 2. Auflage. 2012
- Mazlami, G.; Cito, J.; Leitner, P.: „Extraction of Microservices from Monolithic Software Architectures“ IEEE 24th International Conference on Web Services, o.O. 2017.
- Martin, R.C.: “The single responsibility principle” The Principles, Patterns, and Practices of Agile Software Development, pp. 149–154, 2002.
- Joseph Kruskal: On the shortest spanning subtree and the traveling salesman problem. In: Proceedings of the American Mathematical Society. 7 (1956), S. 48–50
- Mo, Cai, Kazman et al.: Decoupling Level: A New Metric for Architectural Maintenance Complexity. ICSE 2016.



Ablaufdiagramm des Designs nach Oestereich



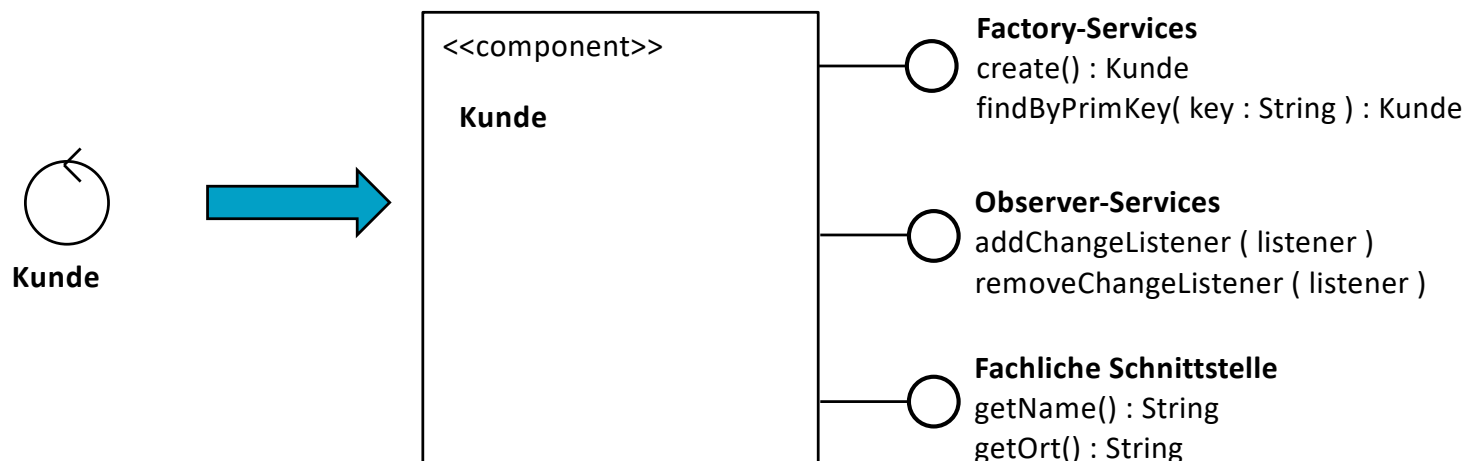
Schichtenmodell nach Oestereich



Quelle: Oestereich, 2009, S. 176



- Abgrenzung Komponente vs. Klasse: „Komponenten austauschbar“ (S. 81)
- Empfehlungen für die Entwicklung von Komponenten:
 - Die Schnittstellen einer Komponente sollten möglichst nur primitive Typen (Integer, String, etc.) enthalten
 - Für die fachliche Kompatibilität sind entsprechende fachliche Standards notwendig
 - Assoziationen zwischen Komponenten sollten mit Benachrichtigungs- und Beobachtungsmechanismen ersetzt werden (→ Observer Pattern)
 - Geringe Kopplung – hohe Kohäsion
- Auch Entity-Klassen (Fachklassen) werden als Komponenten realisiert. Struktur:





- Dialoge
 - Schnittstelle zu den Anwendern
 - Präsentation von Informationen
 - Nehmen Eingaben entgegen
 - Aus mehreren Elementen zusammengebaut (Eingabefelder, Schaltflächen)
- Dialogsteuerung
 - Zuständig für die (clientseitige) Anwender-System Kommunikation
 - Steuerung des Dialogs
 - Angemessene Darstellung und Formatierung der Informationen
 - Eingaben werden formal bearbeitet, nicht inhaltlich
 - Delegation an unterliegende Schichten bei bestimmten Eingaben (z.B: „OK“)
- Dialog-Agent
 - Controller (Steuerungsobjekt), das als dialogspezifischer Stellvertreter des Clients auf dem Server fungiert
 - Schnittstelle zwischen Client und den fachlichen Komponenten
 - Ziel: Optimierung der Interaktion zwischen Client und Server

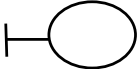


- Fachliche Komponenten
 - Repräsentieren die fachliche Anwendungsdomäne
 - Fachklassen sind in der Regel persistierbar
 - Zusammensetzung meist aus Entity und Controller-Klassen
- Komponente für die Anwendungsfallsteuerung (Use Case Controller)
 - Teil der Fachkomponenten
 - Steuert die im Rahmen eines Use Case (Anwendungsfalls) festgelegten Abläufe (→ pro Use Case eine Komponente)
 - Moderiert den Ablauf zwischen den fachlichen Klassen (Entities) und den übrigen Komponenten
 - Zuständig für die Anzeige von Dialogen
- Komponente für die Workflow-Steuerung
 - Initiiert, überwacht und steuert die Abarbeitung eines Geschäftsprozesses aus fachlicher Sicht (→ pro Geschäftsprozess eine Komponente)
 - Bedeutung vor allem für langfristige Prozesse, wo der Zustand der Bearbeitung für die Zeit einer Unterbrechung zwischen gespeichert werden muss.



- Die Referenzarchitektur von Oestereich bildet einen Rahmen auch für die Entwicklung von heutigen Komponentenarchitekturen
- In Ergänzung mit den Methoden von Brügge und Dutoit eine solide Methodik zur Entwicklung von objektorientierten (Komponenten-)Architekturen
- Dennoch: die Entwicklung von heutigen Komponentenmodellen ist zum Teil weiter geschritten, die Methoden wurden (noch) nicht angepasst
- Für manche Elemente gibt es auch heute noch keine (direkte) Entsprechung
- Keine (bis geringe) Berücksichtigung von Qualitätsanforderungen



- Unterteilung der analysierten Objekte (Klassen...) in Typen ermöglichen eine erste Einteilung und Zuordnung der Konzepte auf das spätere System (= die Software-Architektur!)
- **Entity-Objects** 
 - Repräsentieren fachliche Domänen-Objekte (Geschäftsobjekte), die persistent im System gehalten und gepflegt werden
 - Zuordnung: Basis für konzeptuelles Datenmodell (→ Datenbank)
- **Boundary-Objects** 
 - Repräsentieren Schnittstellen zwischen dem System und externen Akteuren
 - Akteur Endanwender: Benutzerschnittstelle (z.B. GUI)
 - Akteur System: Technische Schnittstelle zu externen Fremdsystemen
- **Control-Objects** 
 - Zuständig für die Koordination zwischen Boundary und Entity-Objects
 - Prozessartigen Charakter, oft repräsentiert durch einen Use Case

Motivation Datentransferobjekt (Data Transfer Objekt)



- Regel: Über Schnittstellen von Schichten sollten kein Entity weitergegeben werden!
 - Entity könnte interne Informationen oder Referenzen auf andere Objekte enthalten, die für eine Schicht (und somit wohlmöglich für den Anwender) nicht zugänglich sein dürfen
- Lösung: Data Transfer Object (Muster nach Fowler (<http://martinfowler.com>)):

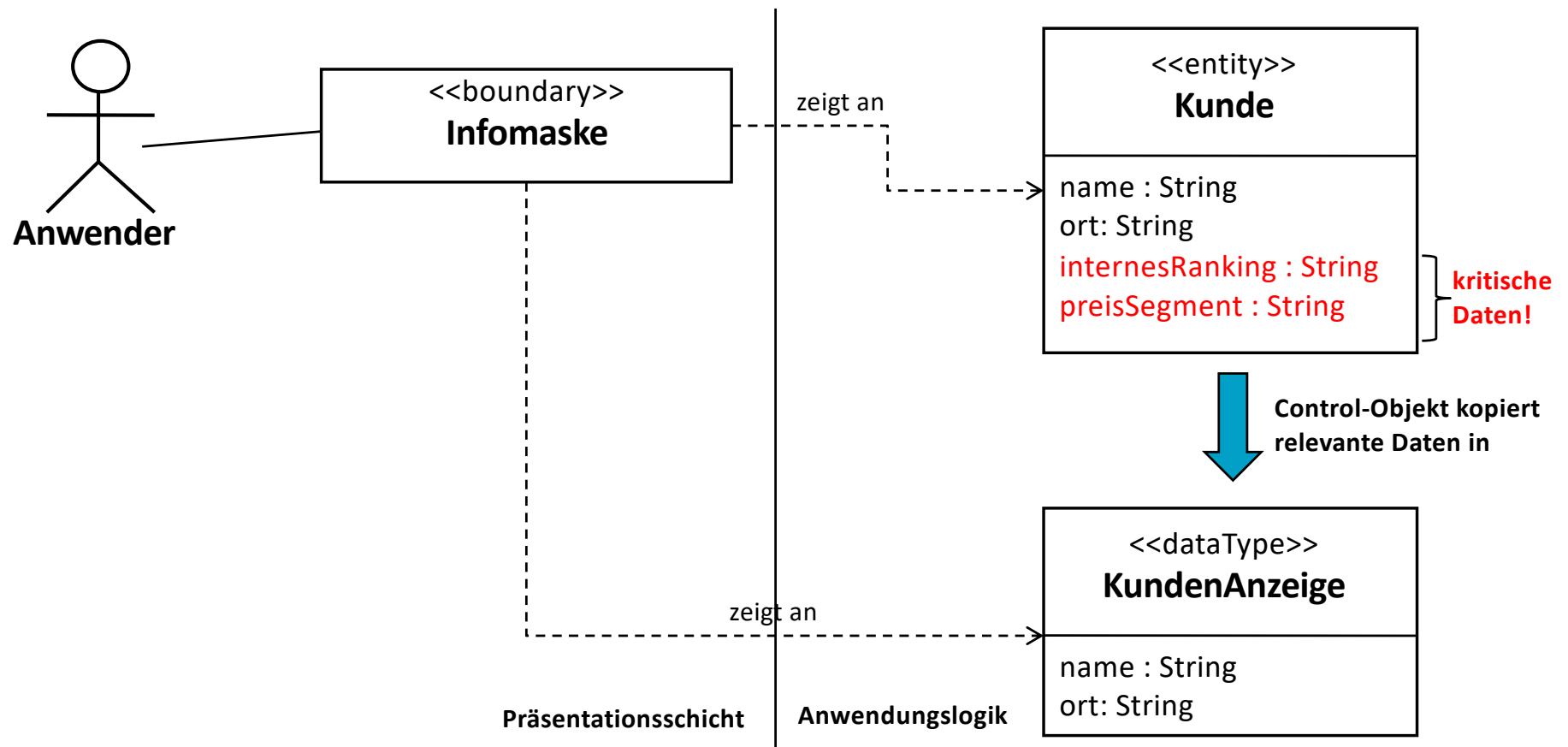
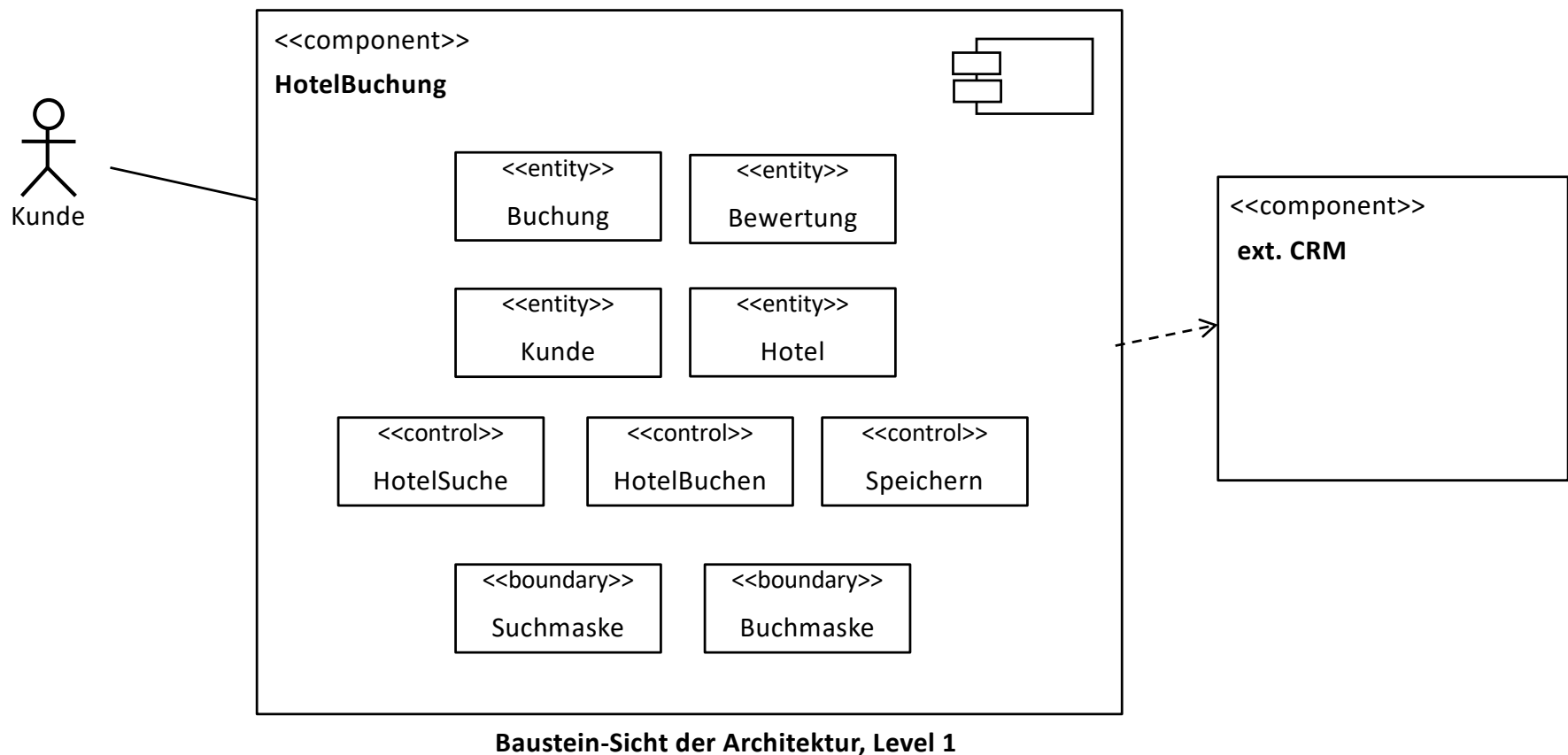


Abbildung typisiertes Objektmodell auf eine Architektur



- Das typisierte Objektmodell ist auf eine Architektur abzubilden
- Vorgehen: schrittweise (iterative) **Dekomposition** der Architektur, Abbildung der Objekte auf konkrete Architektur-Elemente
- Darstellung: **Whitebox** (interne Details werden modelliert)





- Das Control-Objekt wird in Anlehnung an das Pattern MVC (Gamma, 1995) häufig in zwei Klassen geteilt:
 - zur Erstellung und **Navigation** von Boundary-Klassen auf der Präsentationsschicht (Dialog-Steuerung (Oesterreich, 2009))
 - für die Darstellung der **Applikationslogik** (Ablauf des Use Cases) auf der Applikationsschicht (Teil des Modells)

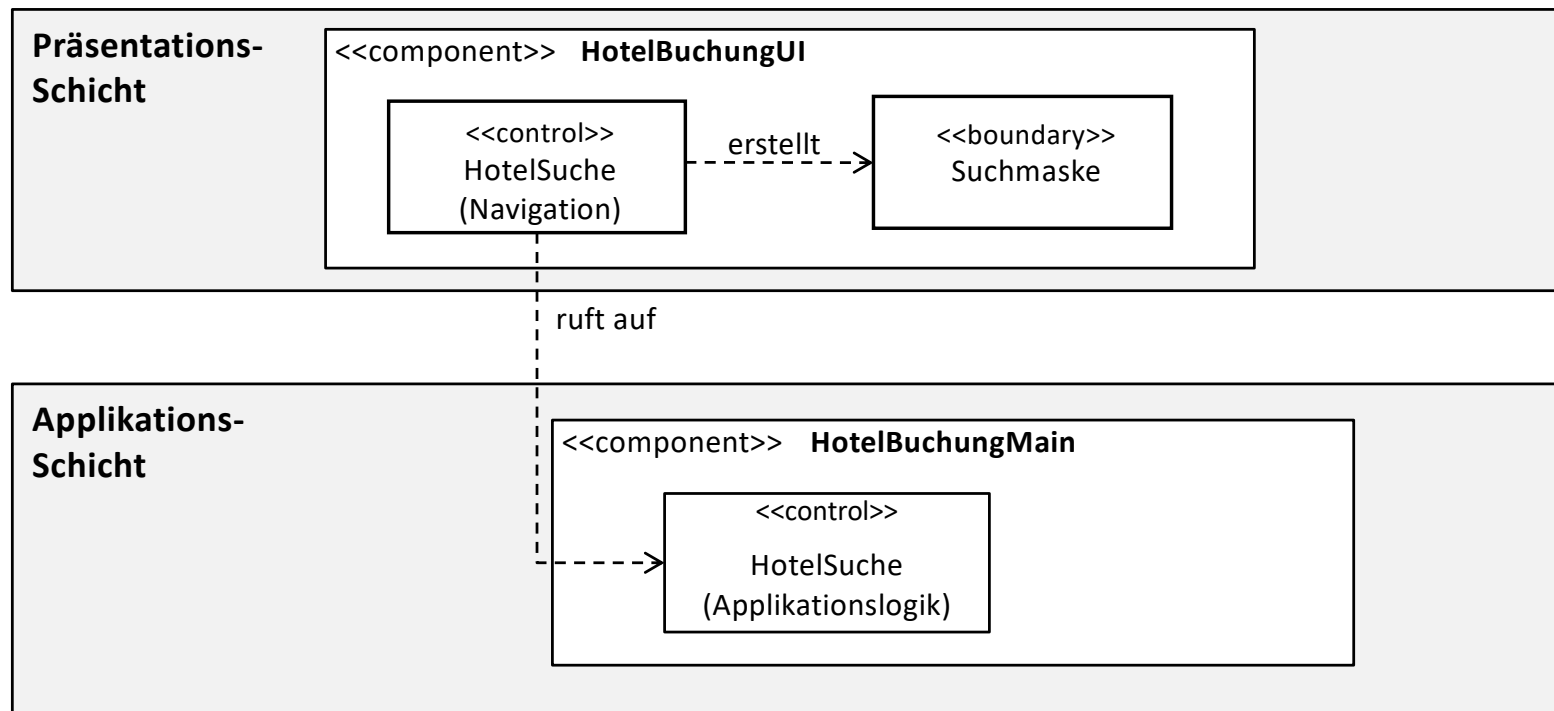
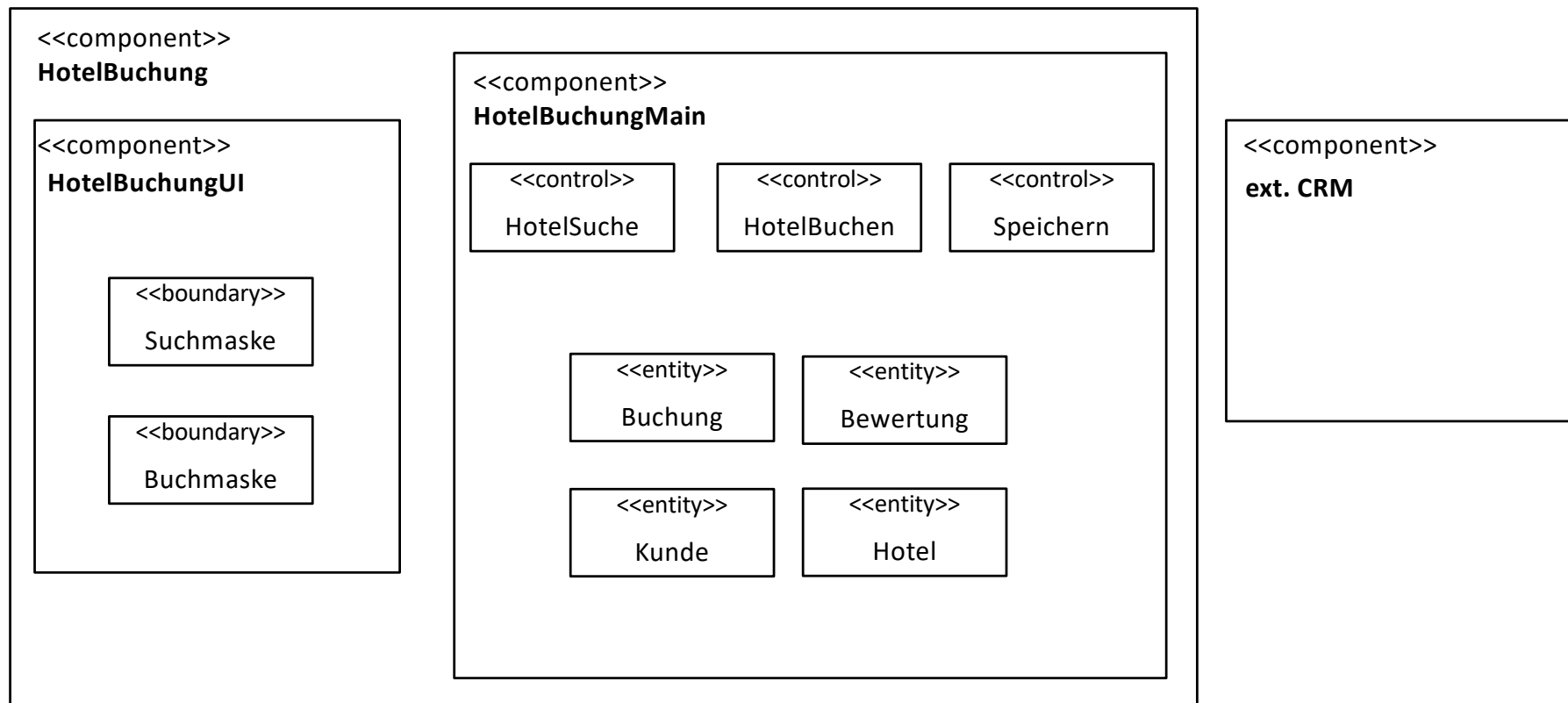


Abbildung typisiertes Objektmodell auf eine Architektur



- Das typisierte Objektmodell ist auf eine Architektur abzubilden
- Vorgehen: schrittweise (iterative) **Dekomposition** der Architektur, Abbildung der Objekte auf konkrete Architektur-Elemente



Baustein-Sicht der Architektur, Level 2

Erweiterung des Architektur-Entwurfs



- Die Betrachtung und Modellierung von dynamischen Aspekten (Laufzeitsicht) ist auch hilfreich für die Identifikation von weiteren Klassen und Methoden
- Beispiel: Modellierung der Interaktionen mit einem Sequenzdiagramms:

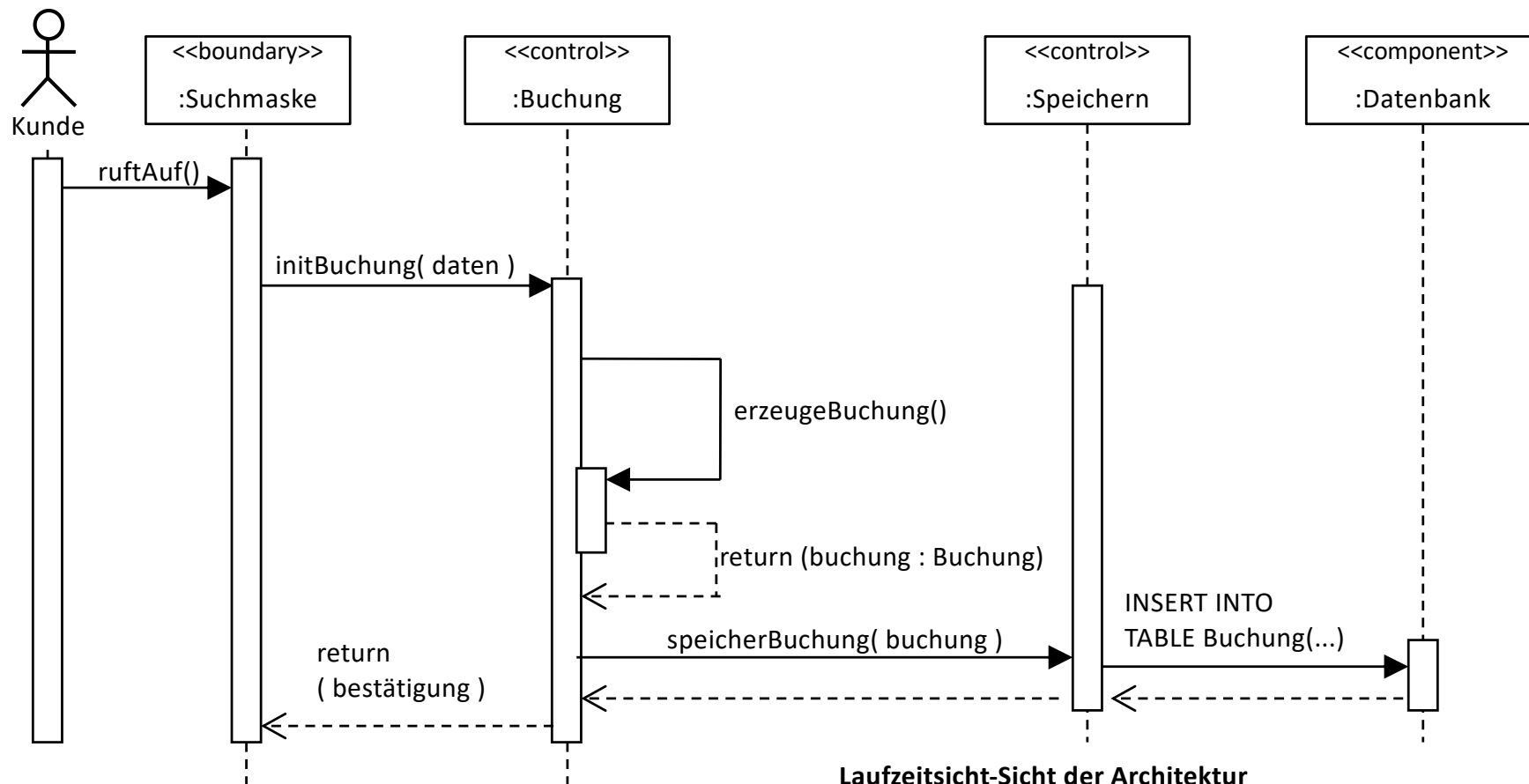
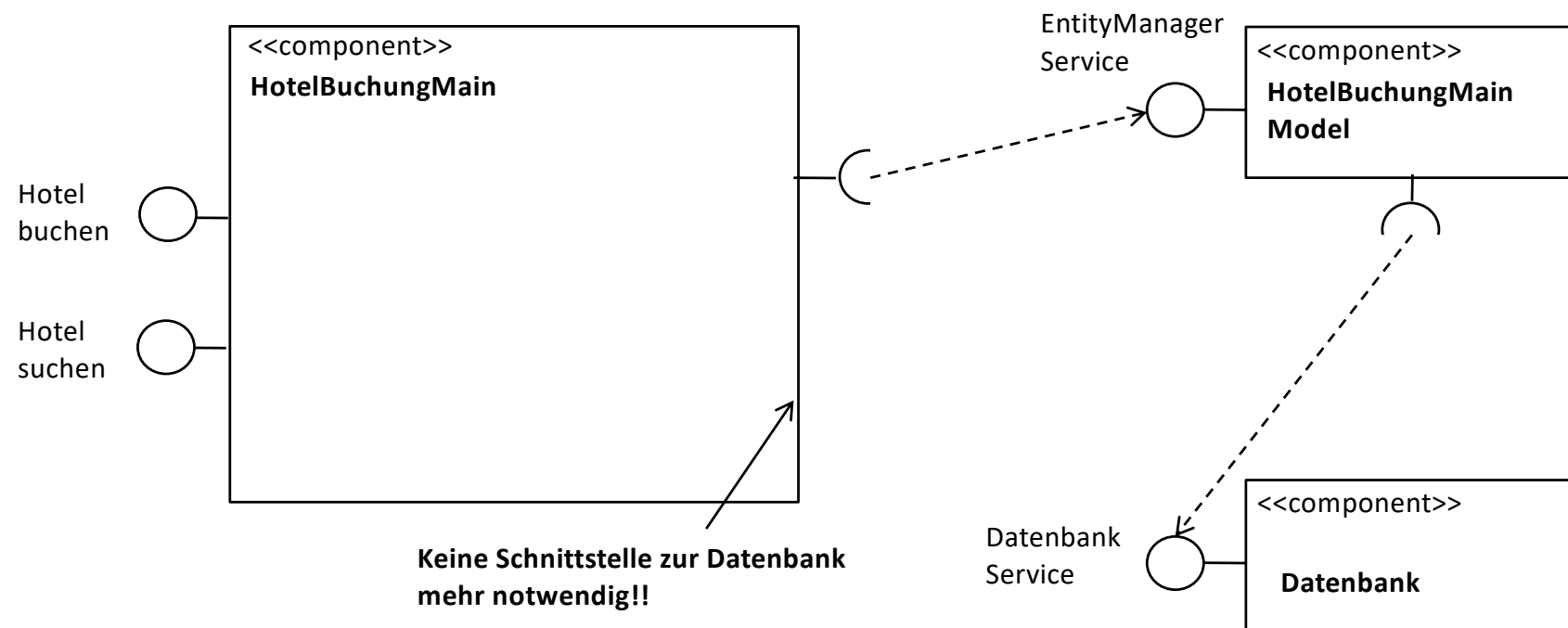


Abbildung Objektmodell auf eine Architektur



- Weiteres Vorgehen: Festlegung der Abhängigkeiten zwischen den Subsystemen
- Beispiel mit EntityManagerService für den geregelten Zugriff auf Entities:



Baustein-Sicht der Software-Architektur der Firma WirSchiffenDas GmbH & Co KGaA

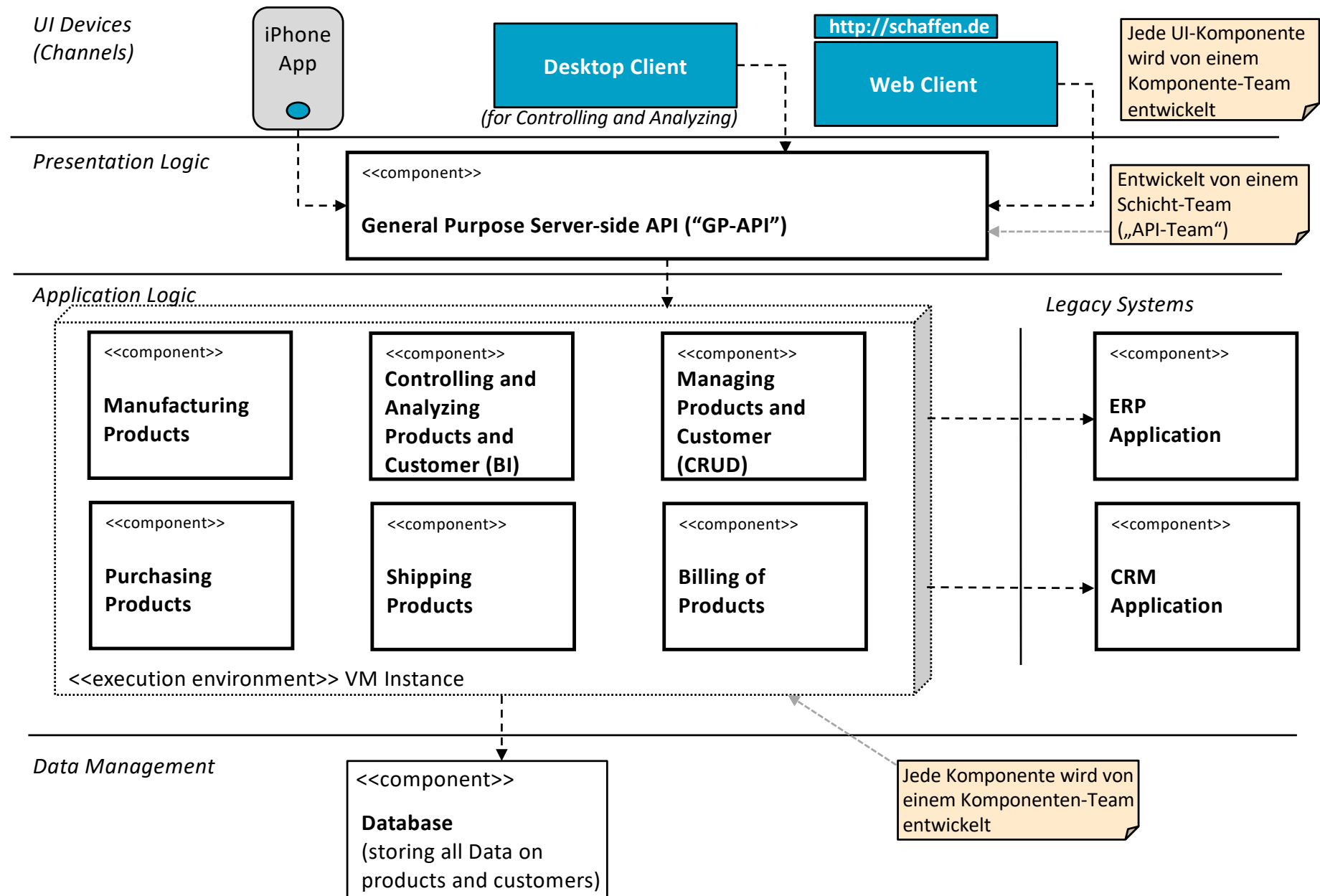
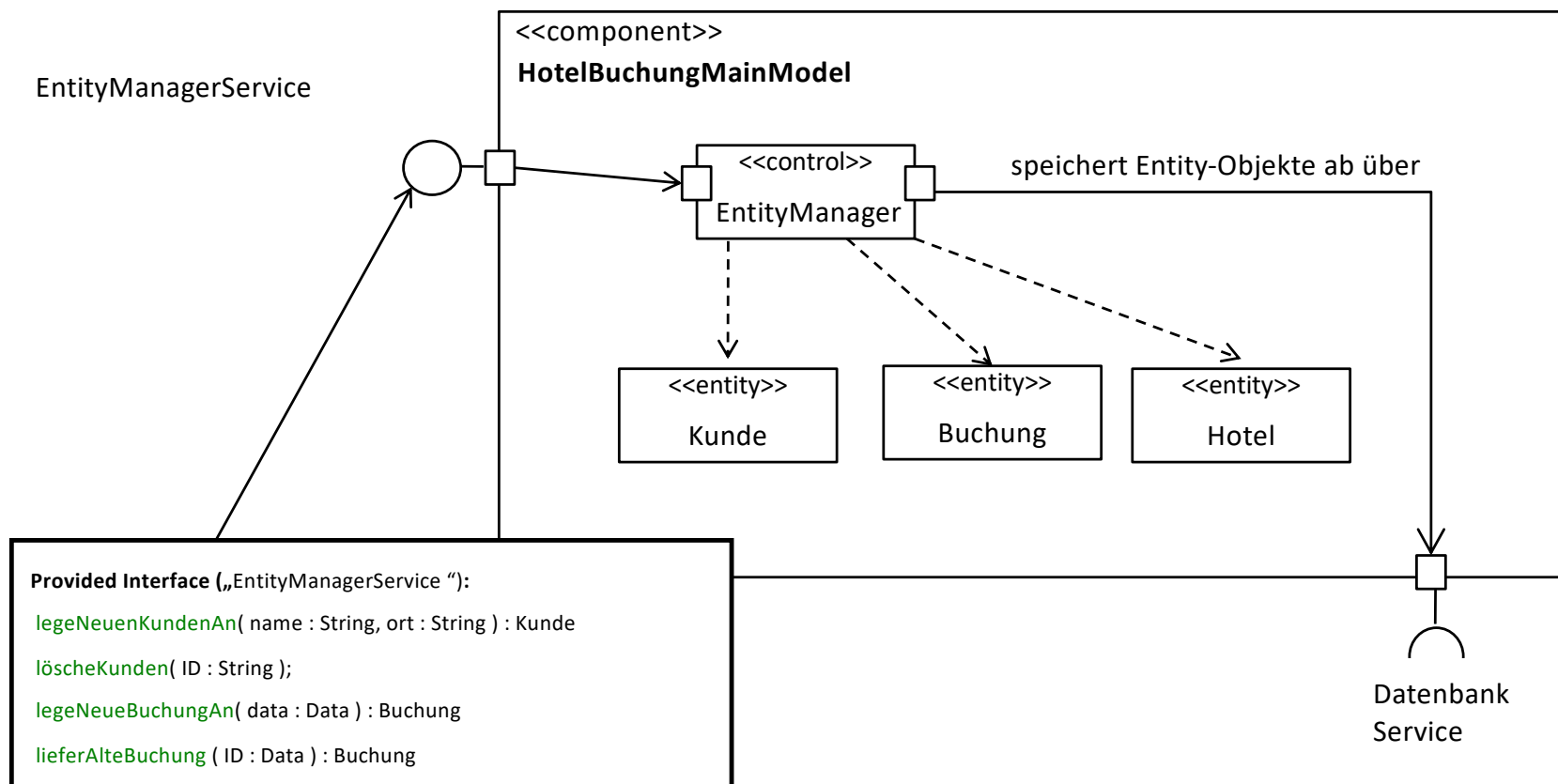


Abbildung Objektmodell auf eine Architektur

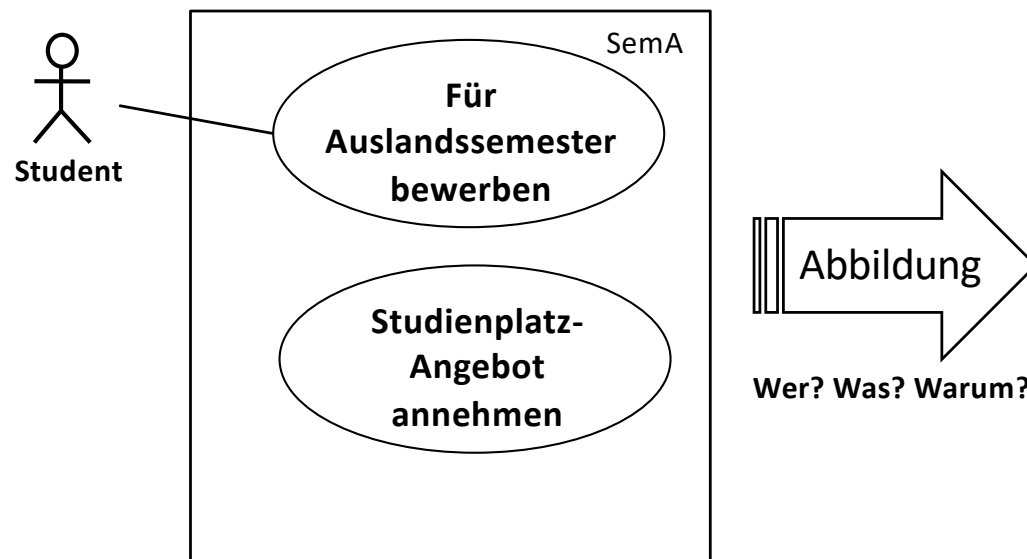


- Der Zugriff auf Entity-Objekte kann über eine dedizierte Schnittstelle erfolgen (z.B. EntityManagerService).
- Vorteil: kontrollierter, zentraler und objektorientierter Zugriff auf das Modell, weniger Code-Redundanz für DB-Zugriff





- Die Anforderungsartefakte Use Case und User Story lassen sich im Lebenszyklus aufeinander abbilden (Rau, 2016)
- Abbildung von Use Cases auf Epics (1:1) sowie Zerlegung der Use Cases (der Epics) in kleinere User Stories nebst Mehrwert (1:n):



Use Case: Für Auslandssemester bewerben

Epic: Der Student will seine Bewerbung für sein Auslandssemester im System erfassen, damit er im Auswahlprozess sachgerecht berücksichtigt werden kann

User Stories:

1.1 Der Student will sich an mindestens drei Partnerhochschulen bewerben, um dabei seine Präferenzen als Input für den Auswahlprozess eingeben zu können

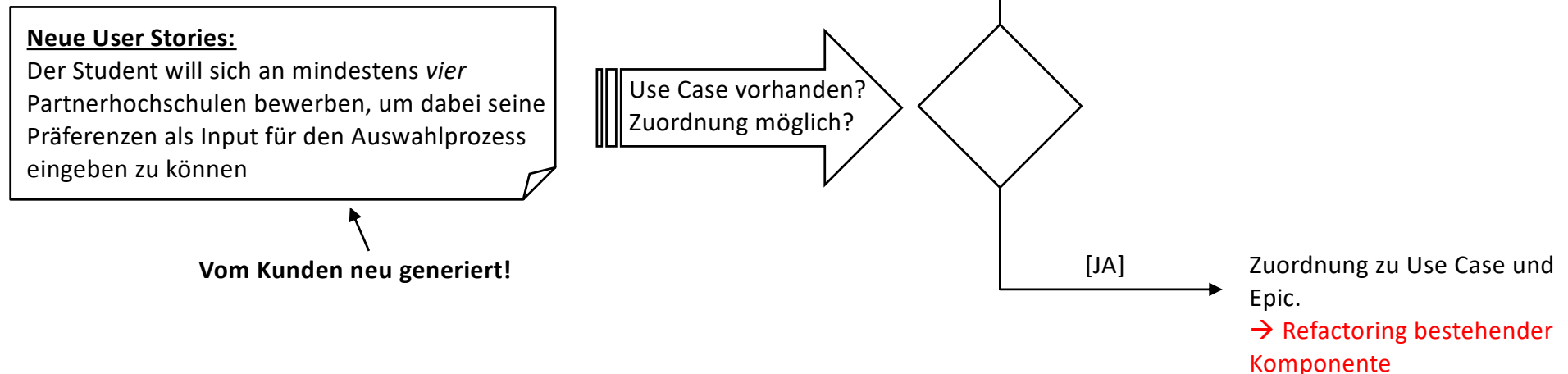
1.2 Der Student will bei seiner Bewerbung einsehen, wie viele Plätze bei den Hochschulen bereit stehen, um seine Chancen zu erhöhen beim Auswahlprozess

...

Beispiel gekürzt übernommen aus (Rau, 2016), Kapitel 2.3



- Die Anforderungsartefakte Use Case und User Story lassen sich im Lebenszyklus aufeinander abbilden (Rau, 2016)
- Kommen im Verlauf des Projekt **neue User Stories in das Backlog**, so muss geprüft werden, ob dazu ein passender Use Case und ggf. eine Komponente vorhanden ist
- Schematische Darstellung (in Anlehnung an UML):

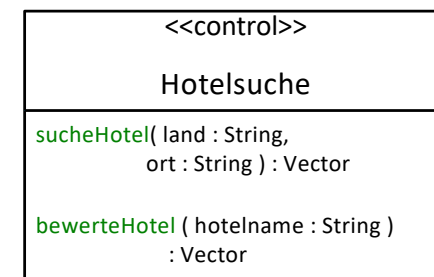
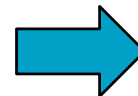




- Abbildung der fachlichen Methoden in die Control-Objekte (nicht Entity!)
 - Identifikation der Methoden z.B. durch die Abbott-Methode (Verben!)
 - **Maßgeblicher Treiber** zur Identifikation einer Komponente
- Control-Objekte sind **zustandslose** Services (Evans, 2004)
 - Zustandslosigkeit: ein Client kann jede Instanz des Service verwenden, ohne Berücksichtigung der (client-basierten) Historie einer bestimmten Instanz
 - Falls client-bezogener Zustand notwendig: Verwendung eines globalen Zustands (Datenbank!)
 - Fördert die Skalierbarkeit und die Anpassbarkeit der Architektur

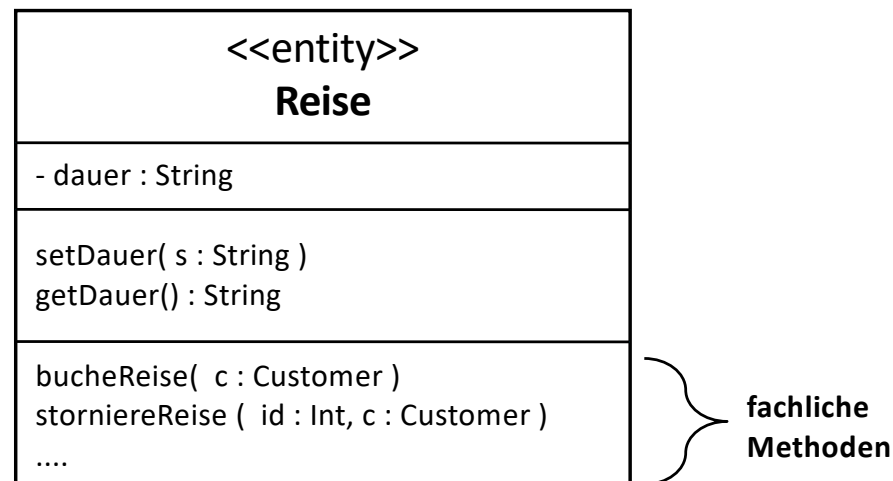
Der Anwender soll nach Hotels in der Suchmaske **suchen** können. Als Suchkriterien gibt er das Land und den Ort an.
...
In der Ausgabe soll der Anwender ein gefundenes Hotel **bewerten** können

Auszug aus Use Case „Hotel Suchen“





- Auf dem ersten Blick scheint es sinnvoller (weil „objektorientierter“), auch fachliche Methoden im Kontext eines Entity mit den dortigen Attributen zu vereinen, anstelle diese in einem Control auszulagern.
- Beispiel Reise:



- Dies wird von vielen (z.B. Fowler, 2003) auch als bevorzugter Modellierungsansatz empfohlen (**Rich Domain Model**) → **Vorteil?!**
- Aber: das umstrittene **Anemic Domain Model** (Entity enthält nur Daten) hat sich in der Industrie durchgesetzt (geprägt durch Frameworks basierend auf Sprachen wie Java oder .NET / C#).